



**Artificial Intelligence (AI):** Field of computer science that solves problems requiring human-like intelligence.

- Examples: image creation, image recognition (car recognizing stop sign), speech-to-text (captions), adaptive learning.

**How AI works:**

- Uses **training data sets** (e.g., fruits: apples, bananas, peaches).
- A **model** is trained by a data scientist.
- Model learns patterns and classifies new data (e.g., new apple image recognized as apple).
- Algorithms like **classification** group similar items together.

**History of AI:**

- **1950s:** Alan Turing → Turing Test (machine fools human in conversation). John McCarthy → coined term “AI.”
- **1970s:** Expert systems (e.g., MYCIN for bacteria diagnosis).
- **1990s:** Machine learning and data mining. Computers powerful enough to handle data.
- **1997:** IBM’s Deep Blue defeats chess champion Garry Kasparov.
- **2010s:** Deep learning revolution with neural networks.
- **2016:** Google AlphaGo defeats Go champion Lee Sedol (Go more complex than chess).

**Modern AI applications:**

- Virtual assistants, autonomous vehicles, healthcare diagnostics, chatbots.
- Doctors use AI to detect tumours.
- Developers use AI co-pilots for coding.
- Businesses automate invoice processing with **Intelligent Document Processing (IDP):**
  - Uses computer vision, deep learning, NLP to extract data (seller, buyer, items, prices) from documents.

**Categories of AI:**

- **AI:** Overall field.
- **Machine Learning:** Uses data to train models.
- **Deep Learning:** Uses neural networks.
- **Generative AI (Gen AI):** Creates new content (ChatGPT for text, DALL·E for images).

**Real-world examples:**

- Siri transcribes voice.
- Google Translate breaks language barriers.
- Fraud detection systems catch suspicious transactions.
- AI helps automate business processes.

**Cloud Computing Definition:** On-demand delivery of computing power, database storage, applications, and IT resources.

- Key idea: **on-demand** → you get resources instantly when needed.
- **Pay-as-you-go pricing** → only pay while using, stop paying when done.
- **Flexibility of Resources:**
  - Choose server size (big, small, many, few).
  - Provision instantly, not hours or days later.
  - Access via easy web interface (e.g., AWS console).
- **Traditional IT vs Cloud:**
  - Traditional: build own data center, buy servers upfront.
  - Cloud: rent servers from provider, pay only for usage.
  - Examples: Gmail (email cloud service), Dropbox/Google Drive (cloud storage), Netflix (video streaming built on AWS).

- **Types of Cloud:**
  - **Private Cloud:** Used by one organization, more control and security (e.g., Rackspace).
  - **Public Cloud:** Shared, operated by providers like AWS, Azure, Google Cloud.
  - **Hybrid Cloud:** Mix of private + public. Sensitive data stays on-premises, flexible workloads go to cloud.
- **Five Characteristics of Cloud Computing:**
  - On-demand self-service (no human intervention needed).
  - Broad network access (resources available anywhere).
  - Multi-tenancy & resource pooling (many customers share infrastructure securely).
  - Rapid elasticity & scalability (scale up/down quickly).
  - Measured service (pay only for actual usage).
- **Six Advantages of Cloud Computing:**
  - Trade **CAPEX** (buying hardware) for **OPEX** (renting resources).
  - Benefit from economies of scale (AWS lowers costs as usage grows).
  - Stop guessing capacity (scale automatically based on demand).
  - Increased speed & agility (launch servers/software instantly).
  - Stop spending on running/maintaining data centers.
  - Build global applications quickly using worldwide infrastructure.
- **Problems Solved by Cloud:**
  - Flexibility (adapt resources easily).
  - Cost-effectiveness (pay only for use).
  - Scalability & elasticity (expand or shrink as needed).
  - High availability & fault tolerance (multiple data centers worldwide).
  - Agility (rapid development, testing, deployment).

Real-world example: A startup can launch a global app in minutes using AWS servers instead of buying expensive hardware, making cloud computing a “no brainer” for modern IT.

### Types of Cloud Computing:

- **Infrastructure as a Service (IaaS):** Provides raw building blocks like networking, computers, and storage. Offers high flexibility, similar to building with LEGOs. Example: **Amazon EC2**, Google Cloud, Azure, Rackspace, Digital Ocean, Linode.
- **Platform as a Service (PaaS):** Removes need to manage infrastructure. Focus only on deploying and managing applications. Example: **AWS Elastic Beanstalk**, Heroku, Google App Engine, Windows Azure.
- **Software as a Service (SaaS):** Fully managed product by provider. User only consumes the service. Examples: **AWS Rekognition**, Gmail, Dropbox, Zoom.

### Comparison of Responsibilities:

- **On-premises:** You manage everything (applications, data, runtime, middleware, OS, virtualization, servers, storage, networking).
- **IaaS:** You manage applications, data, runtime, middleware, OS. Provider manages virtualization, servers, storage, networking.
- **PaaS:** You manage only applications and data. Provider manages everything else.
- **SaaS:** Provider manages everything.

### AWS Pricing Fundamentals:

- **Compute:** Pay for exact compute time used.
- **Storage:** Pay for exact amount of data stored.
- **Networking:** Pay only when data leaves the cloud. Data entering the cloud is free.

### Key Benefit:

- Traditional IT required upfront hardware purchases and capacity planning.
- Cloud computing shifts to **pay-as-you-go**, reducing costs and providing flexibility.
- Real-world example: Using Gmail, Dropbox, or Zoom means you’re consuming SaaS without worrying about servers or infrastructure.

## AWS Cloud History:

- **2002:** AWS started internally at Amazon.com to externalize IT infrastructure.
- **2004:** First public service launched → **SQS (Simple Queue Service)**.
- **2006:** Expanded with **SQS, S3 (storage), EC2 (compute)**.
- **2007:** Later expanded globally beyond the US to Europe and other regions.
- Today, major companies like **Dropbox, Netflix, Airbnb, NASA** run on AWS.

## AWS Market Position:

- AWS is a leader in Gartner's Magic Quadrant for over 13 years.
- Revenue: **\$90 billion (2023)**.
- Market share: **31% in Q1 2024**.
- Over **1 million active users**.
- Competitors: Microsoft Azure (25%), Google Cloud, others.

## What You Can Build on AWS:

- Sophisticated, scalable applications across industries.
- Use cases: enterprise IT migration, backup/storage, big data analytics, website hosting, mobile/social app backends, gaming servers.
- Examples: Netflix streaming, McDonald's apps, Activision gaming servers.

## Global Infrastructure:

- **Regions:** Clusters of data centers worldwide (e.g., US-east-1, EU-west-3). Services are region-specific.
- **Choosing a Region:** Factors include compliance (local laws), latency (close to users), service availability, and pricing differences.
- **Availability Zones (AZs):** Each region has 3–6 AZs (e.g., Sydney → ap-southeast-2A, 2B, 2C). Each AZ = one or more data centers with redundant power/networking. AZs are isolated from disasters but connected with high-bandwidth, low-latency links.
- **Points of Presence (Edge Locations):** Over **400 points of presence in 90 cities across 40 countries**. Used for content delivery with lowest latency (e.g., CloudFront).

## Service Scope:

- **Global Services:** IAM (identity), Route 53 (DNS), CloudFront (CDN), WAF (firewall).
- **Region-Scoped Services:** EC2, Elastic Beanstalk, Lambda, Rekognition.
- To check service availability, AWS provides a **region table**.

## Creating an AWS Account:

- Go to AWS website → click **Create account**.
- Enter **root user email address** and **account name**.
- Verify email → set password (must be at least 8 characters, include 3 of: uppercase, lowercase, numbers, special characters).
- Keep password safe.
- **Account Plan Options:**
  - **Free plan:** No credit card required upfront. You get ~\$100 credits (up to \$200). If credits run out, account closes automatically.
  - **Paid plan:** Requires credit card. Suitable for production workloads. If credits run out, charges go to card.
  - For learning, choose **free plan**.
- **Contact Information:**
  - Enter personal details (name, address).
  - Credit card required even for free plan (for fraud prevention). \$1 temporary charge for verification, refunded immediately.
  - Provide phone number for identity confirmation.
- **Support Plan:**
  - Choose **Basic Support** (free).
  - Complete signup → account activation email arrives.

- **Login Process:**
  - Sign in with **root user email**.
  - Enter password.
  - If prompted with new sign-in options, skip for now.
  - Access AWS Management Console.
- **Outcome:**
  - Account created successfully.
  - You can now explore AWS services safely with free credits.

Real-world note: even free accounts require a card, but you won't be billed unless you upgrade or exceed credits.

- **AWS Console Home Overview:**
  - Top right corner → **Region selector**. Example: Northern Virginia (US-east-1).
  - Best practice: choose a region geographically close to you for lower latency (e.g., Ireland for Europe, Cape Town for Africa).
  - You don't need to be physically in that region to use it.
- **Console Features:**
  - Shows **recently visited services** (initially empty).
  - Bottom section → account health info, cost/usage details, tutorials.
  - Layout may change over time, but core functions remain.
- **Accessing Services:**
  - Top left → **Services menu**. Browse alphabetically or by category (e.g., compute, storage).
  - **Search bar** → type service name (e.g., Route 53) to find services, features, documentation, blogs, and knowledge articles.
- **Global vs Regional Services:**
  - Example: **Route 53** is a **global service** (no region selection needed).
  - Example: **EC2** is **region-scoped** (resources differ by region).
  - Important: stay in the same region during tutorials to avoid confusion.
- **AWS Global Infrastructure:**
  - AWS provides a **global infrastructure map** showing regions and services.
  - **Regional Services Table** → check which services are available in each region.
  - If a service isn't available in your chosen region, switch to another region in the console.

### Shared Responsibility Model:

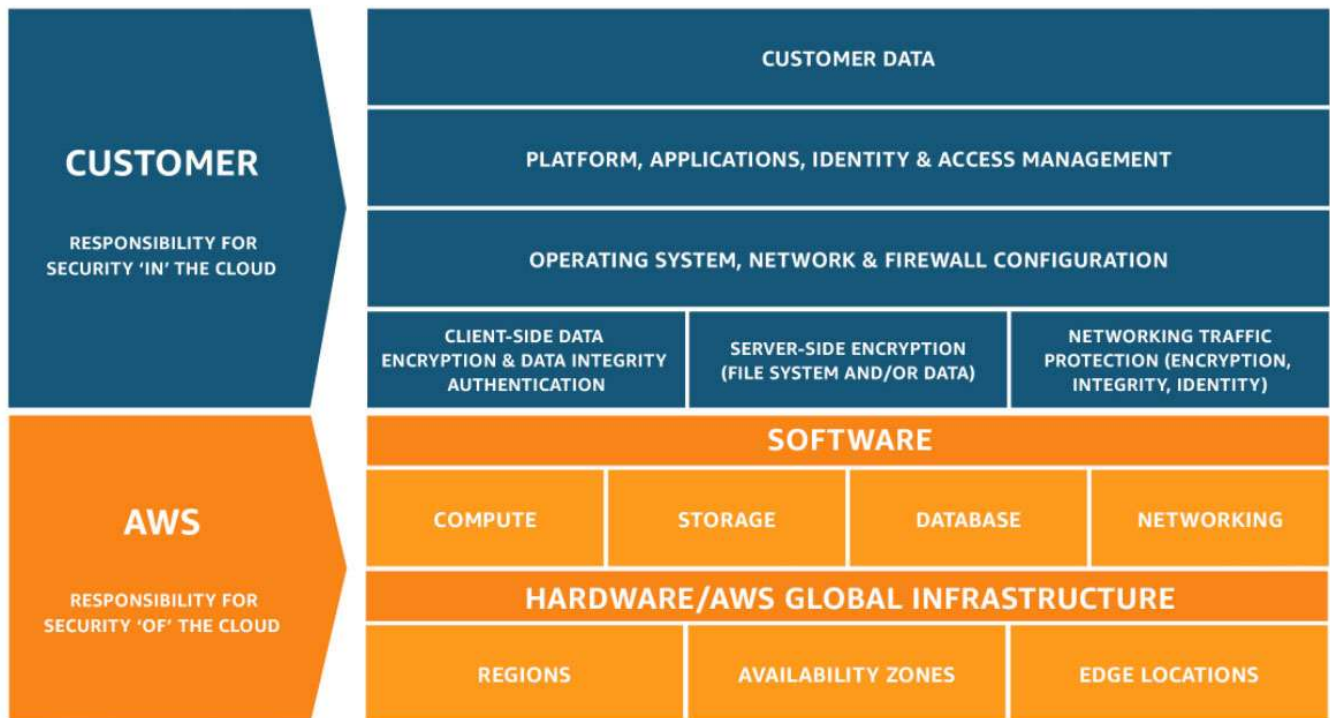
- **Customer responsibility (security *in* the cloud):**
  - How you configure and use services.
  - Protecting your data.
  - Managing operating systems.
  - Setting up networks and firewalls.
- **AWS responsibility (security *of* the cloud):**
  - Physical infrastructure (servers, data centers).
  - Hardware and software maintenance.
  - Internal security of their systems.

### Exam Relevance:

- Certified Cloud Practitioner exam will ask which responsibilities belong to you vs AWS.
- Important to distinguish between **in the cloud** (your responsibility) and **of the cloud** (AWS responsibility).

### Acceptable Use Policy:

- No illegal, harmful, or offensive content.
- No security violations.
- No network abuse.
- No email or messaging abuse.



- **Generative AI (Gen AI):**
  - Subset of deep learning → subset of machine learning → subset of AI.
  - Purpose: generate new data similar to training data.
  - Can be trained on text, images, audio, code, video, etc.
  - Example: train on dog photos + cartoon drawings → generate a cartoon dog.
- **Foundation Models:**
  - Large, general-purpose models trained on massive datasets.
  - Cost tens of millions of dollars to train.
  - Can perform multiple tasks: text generation, summarization, image creation, chatbots, Q&A.
  - Examples: **GPT-4o (OpenAI), Meta models, Google BERT, Amazon, Anthropic.**
  - Some are open source (Meta, Google BERT), others commercial (OpenAI, Anthropic).
- **Large Language Models (LLMs):**
  - Built on foundation models, specialized for human-like text.
  - Trained on huge text datasets (books, articles, websites).
  - Tasks: translation, summarization, Q&A, content creation.
  - Example: **ChatGPT (OpenAI, GPT-4o).**
  - Use **prompts** (questions/inputs) to generate responses.
  - **Non-deterministic outputs:** same prompt can produce different answers.
  - Works by predicting next word based on probabilities (e.g., “After the rain, the streets were...” → wet, flooded, slippery, etc.).
- **Generative AI for Images:**
  - Prompt-based image generation (e.g., “blue sky with clouds and Hello written”).
  - Image-to-image transformation (e.g., photo → anime style).
  - Text-from-image (e.g., “How many apples?” → AI counts objects).
  - Popular method: **Diffusion models** (e.g., Stable Diffusion).
    - **Forward diffusion:** add noise to images until unrecognizable. (e.g. cat to noise)
    - **Reverse diffusion:** start with noise + prompt → gradually de-noise → generate new image (e.g., cat with computer).

- **Key Concepts to Remember:**
  - Gen AI combines knowledge to create new outputs.
  - Foundation models are broad and expensive to train.
  - LLMs generate text, are non-deterministic, and rely on probability.
  - Diffusion models generate images by reversing noise.

### Amazon Bedrock Overview:

- Fully managed AWS service for building **generative AI applications**.
- No infrastructure to manage; AWS handles the backend.
- You keep full control of your data → stays within your account.
- **Pay-per-use pricing** model.
- Provides a **unified API** to access multiple foundation models.
- Includes advanced features like **RAG** (Retrieval Augmented Generation), **LLM agents**, and built-in security, privacy, and governance.

### Foundation Models Available in Bedrock:

- Models from multiple providers: **AI21 Labs**, **Cohere**, **Stability.ai**, **Amazon**, **Anthropic**, **Meta**, **Mistral AI**, and more.
- AWS makes a copy of the chosen model for your account.
- You can fine-tune models with your own data → tailored to your use case.
- Your data is never sent back to providers; it remains private.

### How Bedrock Works:

- Central hub of foundation models.
- Interactive playground → test models by asking questions (e.g., “What is the most popular dish in Italy?” → “Pizza and pasta”).
- Knowledge bases (RAG) → fetch external data for more accurate answers.
- Fine-tuning → adapt foundation models with your own datasets.
- Unified API → same interface for all models, simplifying application development.

### Key Benefits:

- Easy experimentation with multiple models.
- Consistent API integration across providers.
- Strong governance and responsible AI features.
- Scales with your needs while keeping data secure.

### Amazon Bedrock Practice Overview:

- From the **Overview page**, you can explore available models.
- **Model Catalog**: filter by provider (Amazon, Anthropic, Cohere, Stability.ai, etc.) or by capability (text, image).
- Clicking a model shows its overview, categories, pricing, and provider details.

### Testing Models:

- Use the **Chat Playground** → options: single prompt or chat.
- Example: select **Amazon Nova Micro** (low-cost model).
  - Prompt: “What is AWS?” → model generates response.
  - Console shows input tokens, output tokens, latency.
- Swap to another model (e.g., **Anthropic Claude Sonnet 4.5**) → different formatting, token usage, latency.
- Some models support **attachments** → upload a file and ask questions about its content.

### Image and Video Playground:

- Choose models like **Nova Canvas 1.0** for image generation.
- Options: image size, number of images, color palette, reference image.
- Example prompt: “Show me a person wearing an AWS backpack.” → generates multiple images.
- Quality varies by model; some outputs may be more accurate than others.

## Key Takeaways:

- Bedrock makes it easy to **discover models** by provider or capability.
- You can **test models interactively** with text prompts or image prompts.
- Each model behaves differently in terms of output style, latency, and features.
- Unified interface simplifies experimentation across providers.

## Amazon Bedrock – Base Foundation Model

### • Choosing Foundation Models:

- Decision depends on model type, performance needs, capabilities, compliance, customization options, licensing, context window size, latency, and whether the model is **multimodal** (handles text, audio, video, images).
- Smaller models → cheaper, faster, but less knowledge. Larger models → more powerful, but costlier.

### • Amazon Titan (AWS model):

- High-performing foundation model from AWS.
- Supports text, images, and multimodal inputs/outputs.
- Can be fine-tuned with your own data.
- Integrated via Amazon Bedrock unified API.
- Use cases: content creation, classification, education.

### • Comparison of Four Models:

| Model            | Provider     | Capabilities                              | Context Window   | Use Cases                                             | Pricing (per 1K tokens) |
|------------------|--------------|-------------------------------------------|------------------|-------------------------------------------------------|-------------------------|
| Amazon Titan     | AWS          | Text (100+ languages), images, multimodal | 8K               | Content creation, classification, education           | Cheapest                |
| Llama-2          | Meta         | Text, dialogue (English)                  | 4K               | Tech generation, customer service                     | Moderate                |
| Claude           | Anthropic    | Text generation, analysis                 | 200K             | Document comparison, forecasting, large context tasks | Expensive               |
| Stable Diffusion | Stability AI | Image generation only                     | 77 Tokens/Prompt | Advertising, media, creative visuals                  | Cost per image          |

### • Key Insights:

- **Claude** has the largest context window (200K tokens) → best for large documents, code bases, or books.
- **Amazon Titan** is versatile, cost-effective, and exam-relevant for AWS certification.
- **Llama-2** is good for dialogue and customer service tasks.
- **Stable Diffusion** is specialized for image generation.
- Pricing varies significantly; cheaper models may be sufficient depending on business needs.

## Different Providers and Models:

- Providers like **Anthropic**, **Amazon**, **DeepSeek**, and **Stability AI** each offer models with unique capabilities.
- Example: **Claude 3.5 Haiku (Anthropic)** → strong for text tasks.
- Example: **Amazon Nova Reel** → specialized for text-to-video or image-to-video generation.
- Key exam point: know what a model *can* and *cannot* do, not which one is “better.”



### Model Comparison in Playground:

- Use **Compare Mode** in Chat/Text Playground.
- Example: **Amazon Nova Micro** vs **Claude 3.5 Sonnet**.
  - Nova Micro → does not support image uploads.
  - Claude Sonnet → supports image inputs, more expensive, slower latency.
- Prompt: “What are the top AWS services?”
  - Nova Micro → shorter list (11 items).
  - Claude Sonnet → longer, more detailed list (20 items).
- Performance metrics differ input tokens, output tokens, latency.
- Formatting also varies (bullet points vs detailed sections).

### Custom Models in Bedrock:

- Three customization techniques:
  - **Reinforcement fine-tuning:** for complex multi-step reasoning, requires reward functions.
  - **Supervised fine-tuning:** adapts pretrained models with labelled data for domain-specific tasks.
  - **Distillation:** creates smaller, faster models by training a “student” model from a larger one.
- Example workflow:
  - Select base model (e.g., **Nova 2 Lite**).
  - Configure hyperparameters.
  - Provide input data (Amazon S3).
  - Define output storage (validation results, logs).
  - Assign service role → Bedrock creates the custom model.

### Fine-tuning a model in Amazon Bedrock:

- Purpose: adapt a copy of a foundation model with your own data.
- Fine-tuning changes the model’s weights → makes it better suited for your specific use case.
- Training data must follow a specific format and be stored in **Amazon S3**.
- Example: fine-tune **Llama 2** with prompt-completion pairs stored in S3 → customized model.

### Methods of Fine-tuning:

- **Supervised fine-tuning:**
  - Uses labelled examples (input + output pairs).
  - Example: Prompt → “Who is Stephane Maarek?” Completion → “Best AWS instructor...”
  - Cheapest option, improves performance on specific tasks.
- **Reinforcement fine-tuning:**
  - Uses prompts + reward function to judge outputs.
  - Model generates multiple responses → scored by reward function.
  - Reward function can be code (objective tasks) or a judge model (subjective tasks).
  - Iterative learning process.
  - Example: chatbot responses scored for empathy and helpfulness using judge model.
- **Distillation:**
  - Large “teacher” model trains a smaller “student” model.
  - Focus: efficiency, speed, cost reduction (up to 75% cheaper).
  - Trade-off: smaller model less accurate but faster and cheaper.

### Costs and Requirements:

- Fine-tuning requires budget and ML expertise.
- Running fine-tuned models is more expensive than base models.
- Pricing options:
  - **On-demand:** pay per token.
  - **Provisioned throughput:** monthly billing for reserved capacity.

## Use Cases:

- Chatbots with specific persona or tone.
- Training with up-to-date or exclusive data (e.g., customer emails, records).
- Domain-specific tasks like categorization or document analysis.
- Improving accuracy for targeted business needs.

## Amazon Bedrock – Evaluating a Model

- **Automatic Evaluation in Amazon Bedrock:**
  - Purpose: evaluate a model's quality and performance rigorously.
  - Built-in task types: **text summarization, question answering, text classification, open-ended text generation.**
  - Use **prompt datasets** (your own or AWS curated) → benchmark questions + benchmark answers.
  - Model generates answers → compared against benchmark answers.
  - A **judge model** evaluates similarity and assigns scores using metrics like **BERTScore, F1**, etc.
- **Benchmark Datasets:**
  - Curated collections designed to test model performance.
  - Measure accuracy, speed, scalability.
  - Detect bias or discrimination quickly.
  - Can use AWS-provided datasets or create custom ones for business-specific needs.
- **Human Evaluation:**
  - Same process as automatic evaluation but humans compare outputs.
  - Evaluators: employees, SMEs, or external reviewers.
  - Metrics: thumbs up/down, ranking, grading scores.
  - More flexible since humans can assess nuance.
- **Automated Metrics to Evaluate an FM:**
  - **ROUGE:** Recall-Oriented Understudy for Gisting Evaluation. Measures overlap of n-grams (ROUGE-N) or longest common subsequence (ROUGE-L). Best for summarization and translation.
  - **BLEU:** Bilingual Evaluation Understudy. Precision-based, penalizes overly short translations. Best for translation tasks. Looks at combination of n-grams (1,2,3,4)
  - **BERTScore:** Uses embeddings to measure semantic similarity (meaning/context). Best for nuanced text comparison. Uses pre-trained BERT Models (Bidirectional Encoder Representation from Transformers) to compare.
  - **Perplexity:** Measures how well a model predicts the next token. Lower = better confidence.
- **Business Metrics for Evaluation:**
  - **User satisfaction:** gather feedback from end users and assess their satisfaction with the model responses.
  - **Average revenue per user (ARPU):** financial impact of GenAI app.
  - **Cross-domain performance:** ability to handle varied tasks.
  - **Conversion rates:** success in achieving desired outcomes.
  - **Efficiency:** cost, resource utilization, speed.

## Evaluation – Hands on

### Model Evaluation in Amazon Bedrock:

- Found under **Inference and Assessment → Evaluations.**
- Two types: **Automatic evaluation** and **Human evaluation.**

## Automatic Evaluation Approaches:

- **Programmatic:**
  - Select a model + task type (e.g., general text generation, summarization, Q&A).
  - Metrics: **Toxicity, Accuracy, Robustness**.
  - Use built-in or custom prompt datasets.
  - Results stored in **Amazon S3**.
- **Model as a Judge:**
  - Another model evaluates outputs.
  - Example: **Claude 3.5 Sonnet** acts as evaluator.
  - Can assess Bedrock models or external inference responses.
  - Metrics: **Helpfulness, Faithfulness**, etc.
  - Requires datasets + permissions.

## Human Evaluation Approaches:

- **AWS Managed Work Team:** AWS provides evaluators.
- **Bring Your Own Team:** Your workforce or SMEs evaluate outputs.
- Can compare up to **two models** side by side (e.g., Nova Pro vs Claude 3.5 Sonnet).
- Task types: general text generation, summarization, Q&A, or custom tasks.
- Metrics: thumbs up/down, ranking, grading scores.
- Humans provide qualitative judgment on which model performs better.

## Key Takeaways:

- Automatic evaluation → faster, scalable, uses metrics or judge models.
- Human evaluation → nuanced, flexible, captures subjective quality.
- Both approaches help ensure models meet standards for accuracy, safety, and usefulness.

## RAG (Retrieval-Augmented Generation):

- Concept: lets a foundation model reference **external data sources** without fine-tuning.
- Workflow:
  - Data stored in **Amazon S3** or other sources.
  - Bedrock builds a **knowledge base** backed by a **vector database**.
  - Data is converted into **embeddings** (numeric representations).
  - When a user asks a question, relevant info is retrieved → combined with the prompt → passed to the model → generates an augmented response.
- Example: Query “Who is John’s product manager?” → Knowledge base retrieves “Jessie Smith” → model responds with “Jessie Smith is the product manager for John.”
- **Vector Databases for RAG:**
  - **AWS options:**
    - **OpenSearch Service:** scalable, real-time search, KNN nearest neighbour search.
    - **Aurora PostgreSQL:** relational database with vector support.
    - **Neptune Analytics:** graph-based queries (GraphRAG).
    - **S3 Vectors:** cost-effective, durable storage with sub-second queries.
  - **External options:** MongoDB, Redis, Pinecone.
  - Embeddings generated by models like **Amazon Titan** or **Cohere**.
- **Amazon Bedrock RAG Data Sources for Knowledge Bases:**
  - **Amazon S3** (files, PDFs, structured/unstructured data).
  - **Confluence, Microsoft SharePoint, Salesforce, webpages** (company sites, social feeds).
  - More integrations expected over time.
- **Use Cases of RAG in Bedrock:**
  - **Customer service chatbot:** answers based on product specs, FAQs, troubleshooting guides.
  - **Legal research assistant:** references laws, case precedents, expert opinions.

- **Healthcare Q&A:** references diseases, treatments, clinical guidelines, patient records.
- **Business intelligence:** dynamic product descriptions, knowledge retrieval for internal teams.

### Knowledge Bases in RAG Practice:

- In Bedrock Console → **Builder Tools** → **Knowledge Bases**.
- Instead of building a full knowledge base, you can try **Chat with your document** to simulate how RAG works.

### Workflow:

- Select a model (e.g., **Anthropic Claude 3.5 Sonnet**).
- Use the default **chat prompt template**: model acts as a Q&A agent, only answering from search results in your uploaded document.
- Upload a document (e.g., *Evolution of the Internet Detailed*).
- Ask a question → model searches inside the document → retrieves relevant chunks → combines with your query → generates an answer.
- Example: “Who and when invented the World Wide Web?” → Answer cites **Tim Berners-Lee, 1989**, with source chunks highlighted from your document.

### Behaviour:

- If the query matches content in the document → model responds with sourced answer.
- If the query is unrelated (e.g., “How to make guacamole?”) → model apologizes, stating no relevant information found.
- This ensures the model only uses **retrieved knowledge** rather than hallucinating.

### Key Concepts Demonstrated:

- **Search step:** retrieves relevant document chunks.
- **Augmented prompt:** combines query + retrieved text.
- **Model response:** generated answer with citations.
- **Guardrails:** prevent answers outside the provided knowledge base.

### Setting up RAG and Knowledge Bases in Bedrock:

- You cannot use the **root user** for this setup. You must create an **IAM user** with **Administrator Access** policy.
- Log in with the IAM user credentials → proceed to Bedrock.
- **Creating a Knowledge Base:**
  - In Bedrock Console → **Knowledge Bases** → **Create Knowledge Base with Vector Store**.
  - IAM permissions: create and use a new service role.
  - Choose a **data source** (Amazon S3, web crawler, Confluence, Salesforce, SharePoint).
  - Example: upload a PDF into an **Amazon S3 bucket**.
    - Bucket names must be globally unique.
    - Upload your file (e.g., *Evolution of the Internet Detailed*).
  - Select the S3 URI as the data source.
- **Embeddings Model:**
  - Choose an embeddings model (e.g., **Amazon Titan Text Embeddings V2**).
  - This converts document chunks into **vector embeddings**.
- **Vector Database Options:**
  - **Amazon OpenSearch Serverless** (scalable, production-ready, but costly).
  - **Aurora PostgreSQL** (relational).
  - **Neptune Analytics** (graph-based RAG).
  - **S3 Vectors** (cost-effective, durable).
  - External: **Pinecone** (offers free tier), MongoDB, Redis.
  - In the example, OpenSearch Serverless was used → ~\$172/month minimum, so deletion after testing is essential.

- **Synchronization:**
  - After creating the knowledge base, sync the S3 data → embeddings stored in vector DB.
  - You can inspect the OpenSearch dashboard:
    - See indexes, vectors, and document chunks.
    - Each chunk of your PDF becomes a vector embedding.
- **Testing the Knowledge Base:**
  - Configure a model (e.g., **Anthropic Claude 3.5 Sonnet**).
  - Ask: “Who invented the World Wide Web?” → retrieves info from the PDF → generates sourced answer.
  - Sources link back to the original S3 document.
  - If you add more documents to S3, sync again to update the knowledge base.
- **Cleanup:**
  - Delete the knowledge base in Bedrock.
  - Then delete the OpenSearch collection to avoid ongoing costs.
  - S3 bucket can remain (low/no cost).

### Tokenization:

- Converts raw text into **tokens** (smaller units).
- Types:
  - **Word-based tokenization** → splits text into words.
  - **Subword tokenization** → splits long words into smaller parts (e.g., *unacceptable* → “un” + “acceptable”).
- Each token gets an **ID**, making it easier for models to process.
- Even punctuation marks (, !) are tokens.
- Example: “Wow, learning AWS with Stephane Maarek is immensely fun!” → split into tokens like *Wow, learning, AWS, Steph, ane*, etc.

### Context Window:

- Defines how many tokens an LLM can consider at once.
- Larger context windows → more coherent, detailed responses.
- Examples:
  - **GPT-4 Turbo**: 128K tokens.
  - **Claude 2.1**: 200K tokens.
  - **Google Gemini 1.5 Pro**: 1M tokens (research up to 10M).
- 1M tokens ≈ 1 hour of video, 11 hours of audio, 30K+ lines of code, or 700K words.
- Trade-off: larger windows need more memory and processing → higher cost.

### Embeddings:

- Converts tokens into **vectors** (arrays of numbers).
- Example: “The cat sat on the mat” → each word becomes a vector of values.
- Vectors capture meaning, syntax, sentiment, and other features.
- Stored in **vector databases** for similarity search (used in **RAG**).
- Words with similar meaning → similar embeddings (e.g., *dog* and *puppy* close together, *house* far away).
- Visualization:
  - **Dimensionality reduction** → reduce 100+ dimensions to 2–3 for human visualization.
  - **Color embeddings** → represent vectors as colors to show similarity.

### Why Embeddings Matter:

- Enable **semantic search** (find meaning, not just exact words).
- Power applications like **knowledge bases**, **chatbots**, and **recommendation systems**.
- Example: searching “dog” retrieves related terms like “puppy” or “animal.”

- **Guardrails in Amazon Bedrock:**
  - Purpose: control interactions between users and **foundation models**.
  - Can filter **undesirable or harmful content**.
  - Example: block food recipes → user asks “Suggest me something to cook” → response: “Sorry, this is a restricted topic.”
- **Capabilities of Guardrails:**
  - **Content filtering:** block topics irrelevant or unsafe for your business.
  - **PII removal:** strip personally identifiable information to protect user privacy.
  - **Privacy enhancement:** ensure sensitive data isn’t exposed.
  - **Reduce hallucinations:** prevent models from inventing false or misleading answers.
  - **Multiple guardrails:** you can layer them for different levels of control.
  - **Monitoring:** track and analyze violations to refine guardrail setup.

Guardrails are essentially **safety nets** in **Amazon Bedrock**, ensuring that generative AI outputs remain aligned with business rules, compliance requirements, and user safety.

#### Guardrails Overview:

- Guardrails in **Amazon Bedrock** let you enforce responsible AI use by filtering inputs/outputs.
- You can customize the **blocked prompt/response message** (e.g., “Sorry, the model cannot answer this question”).
- Multiple guardrails can be applied at once for layered protection.

#### Configuration Options:

- **Content filters:** block harmful categories like hate, insults, sexual, violence, misconduct.
- **Denied topics:** define restricted subjects (e.g., recipes). Add definitions and sample phrases to help the model recognize them.
- **Word filters:** block profanity or custom words/phrases.
- **Sensitive information filters:** mask personally identifiable information (PII) such as emails, phone numbers. Regex patterns can be added for custom formats.
- **Contextual grounding:** reduce hallucinations by checking relevance and grounding responses in verified data.

#### Demo Example:

- Guardrail blocks food recipes → prompt “Suggest me something to cook tonight” → response: “Sorry, the model cannot answer this question.”
- Guardrail masks PII → prompt includes emails → output email draft shows masked addresses while retaining the rest of the content.

#### Testing Guardrails:

- Apply guardrails to models (e.g., **Anthropic Claude Sonnet**).
- Test prompts in **Text Playground** with guardrails enabled.
- Multiple guardrails can be stacked for broader coverage.

#### Amazon Bedrock Agents Overview:

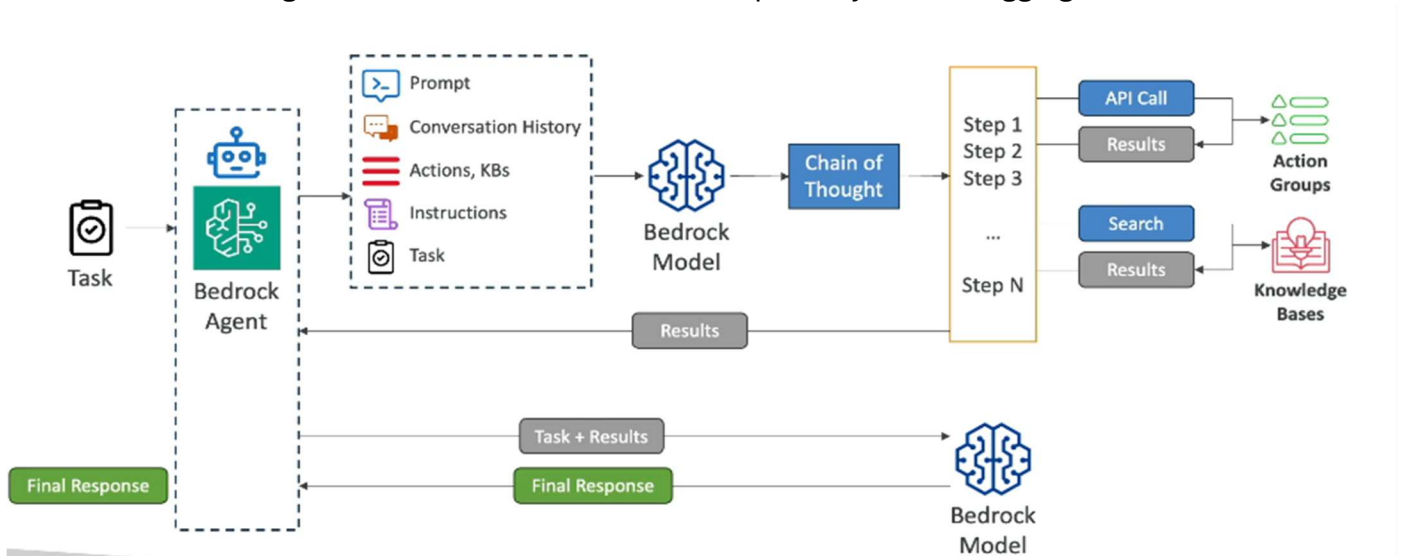
- Agents extend foundation models beyond Q&A into **multi-step reasoning and actions**.
- They can interact with **databases, APIs, and infrastructure**, not just generate text.
- Agents use **action groups** (defined APIs or Lambda functions) to perform tasks.
- They can also reference **knowledge bases (RAG)** for company-specific or real-time data.
- **How Agents Work:**
  - You define the agent’s responsibilities (e.g., purchase history, recommendations, placing orders).
  - Action groups are linked via **OpenAPI schemas** or **Lambda functions**.
  - The agent interprets user prompts, checks available actions + knowledge bases, and generates a **chain of thought** (step-by-step plan).
  - Steps may include calling APIs, querying knowledge bases, or executing Lambda functions.
  - The agent executes these steps, then synthesizes results into a final response for the user.
  - **Tracing** lets you see the agent’s step-by-step execution for debugging.

- **Integration Examples:**

- **Customer purchase agent:** retrieves purchase history, recommends products, places new orders.
- **Lambda-powered agent:** runs serverless functions to update databases or trigger workflows.
- **Knowledge base agent:** references company policies, FAQs, or product specs to answer queries.

- **Behind the Scenes Flow:**

1. User prompt → agent analyzes conversation + available actions.
2. Bedrock model generates **chain of thought** (ordered steps).
3. Agent executes steps (API calls, Lambda, RAG queries).
4. Results synthesized by Bedrock model → final user response.
5. Tracing shows execution details for transparency and debugging.



Amazon Bedrock Agents are essentially **intelligent orchestrators**: they don't just answer questions, they **act**—integrating with APIs, Lambda, and knowledge bases to perform tasks autonomously.

### Amazon Bedrock + CloudWatch Integration:

- **Model Invocation Logging:**
  - Bedrock can send **model invocation logs** (inputs + outputs) to CloudWatch Logs or Amazon S3.
  - Includes text, images, embeddings.
  - Provides a full history of interactions for auditing and debugging.
  - You can analyze logs in real time using **CloudWatch Logs Insights**.
- **CloudWatch Metrics:**
  - Bedrock publishes usage and performance metrics to CloudWatch Metrics.
  - Examples: general invocation counts, latency, token usage.
  - Guardrail-related metric: **content filtered count** → tracks how often guardrails block content.
  - Metrics can be used to create **CloudWatch Alarms** for proactive monitoring (e.g., alert when filtered content spikes, or when latency exceeds thresholds).
- **Benefits of Integration:**
  - **Tracing & Monitoring:** complete visibility into Bedrock activity.
  - **Alerting:** automatic notifications when thresholds are crossed.
  - **Analysis:** real-time insights into model behavior, guardrail effectiveness, and system performance.
  - **Compliance & Governance:** logs provide audit trails for responsible AI usage.

This integration ensures that **Amazon Bedrock** applications are not only powerful but also **observable and accountable**.

## Model Invocation Logging Setup:

- In Bedrock Console → **Settings** → **Model Invocation Logging**.
- Enable logging → captures **metadata, requests, and responses** for all model invocations.
- Choose data types: text, images, embeddings.
- Destination: **CloudWatch Logs, Amazon S3**, or both.
- Specify a **log group name** (e.g., *Bedrock Invocation Logging*).
- Create or assign a **service role** so Bedrock can write logs to CloudWatch.
- For large payloads (>100 KB), logs can be redirected to S3.
- **Creating Log Group in CloudWatch:**
  - If the log group doesn't exist, create it manually in **CloudWatch Logs** → **Log Groups**.
  - Name must match the one defined in Bedrock settings.
  - Configure retention (optional).
- **Testing Logging:**
  - Run a prompt in Bedrock (e.g., "What is AWS?").
  - Logs appear in CloudWatch under the defined log group.
  - Example details captured:
    - **Model ID:** Amazon.Titan-Text-Express-V1.
    - **Region:** invocation region.
    - **Input tokens:** 271.
    - **Output tokens:** response length.
    - **Latency:** e.g., 4,038 ms.
    - **Invocation metadata:** user input, assistant output, configuration.
- **CloudWatch Metrics Integration:**
  - Bedrock publishes metrics by **model ID** and across all models.
  - Ex: **invocation count, invocation latency, content filtered count** (guardrail metric).
  - Metrics can be visualized in CloudWatch dashboards.
  - Build **alarms** to trigger alerts when thresholds are exceeded (e.g., latency too high, excessive filtered content).
- **Benefits:**
  - Full **tracing and monitoring** of Bedrock activity.
  - **Debugging:** analyze token usage, latency, outputs.
  - **Alerting:** proactive notifications for performance or safety issues.
  - **Governance:** audit trail for compliance and responsible AI use.

## Pricing Models in Amazon Bedrock:

- **On-Demand:**
  - Pay-as-you-go, no commitment.
  - Charged per input/output token for text models.
  - Charged per input token for embeddings.
  - Charged per image generated for image models.
  - Works with base models only.
- **Batch Mode:**
  - Multiple predictions processed together.
  - Output delivered as a single file in Amazon S3.
  - Responses delayed (not real-time).
  - Discounts up to 50%.
- **Provisioned Throughput:**
  - Reserve model units for guaranteed throughput.
  - Throughput – max number of input/output tokens processed per minute
  - Required for fine-tuned, custom, or imported models.
  - Not primarily for cost savings—focus is on capacity assurance.



## Costs of Model Adaptation:

- **Prompt engineering:**
  - No retraining, no extra computation.
  - Cheapest way to improve outputs.
- **RAG:**
  - Uses external knowledge bases.
  - No fine-tuning cost, but requires vector database + infrastructure.
- **Instruction-based fine-tuning:**
  - Uses labeled data to steer responses.
  - Requires computation, moderate cost.
- **Domain-specific fine-tuning:**
  - Uses large, unlabeled datasets.
  - High computation, most expensive option.

## Cost-Saving Strategies:

- Use **on-demand** for unpredictable workloads.
- Use **batch mode** for up to 50% discounts (accept delayed results).
- **Provisioned throughput** ensures performance but not cost savings.
- Adjust **temperature, Top-K, Top-P** → affects model behavior, not pricing.
- Choose **smaller models** when possible → usually cheaper.
- Optimize – Number of input and output tokens – Write prompts concisely

## End-to-End Use Case with Bedrock – AI Stylist:

- Up to now, we've explored Bedrock features in the console (playground). But in production, you integrate via **API calls** to build real applications.
- AWS provides a demo called **AI Stylist**, which shows how Bedrock powers a complete workflow.
- **AI Stylist Demo Flow:**
  - **Customer prompt:** "I'm a consultant traveling to New York next week. What outfit should I wear?"
  - **Knowledge bases used:**
    - **Product catalog** (private company data).
    - **Fashion trends** (public dataset).
    - **Order history** (private).
    - **Customer reviews** (private).
    - For this scenario, only product catalog + image generation agent are used.
  - **Agents:**
    - **Product catalog agent** → queries catalog data.
    - **Image generation agent** → creates outfit visuals.
  - **Generated output:** Two looks (business formal + business casual) with AI-generated images and descriptions.
- **Interactive Features:**
  - Ask: "What do people like about the business formal jackets?" → Agent queries **customer reviews** → summarizes 325 reviews (quality, color, fabric).
  - Ask: "Show me specific reviews" → Agent retrieves detailed product reviews.
  - Ask: "What size should I wear?" → Agent checks **order history** → recommends size M.
  - Ask: "Add to cart" → Agent invokes APIs to update shopping cart.
  - Finalize order → Agent completes transaction, integrates with backend systems.
  - Bonus: Agent considers **weather data** to suggest additional items.
- **Key Concepts Demonstrated:**
  - **Knowledge bases** provide structured company + public data.
  - **Agents** orchestrate multi-step tasks (query KBs, call APIs, update cart).

- **Image generation** enhances user experience with visuals.
- **End-to-end integration** shows how console features translate into real applications.
- **Amazon Nova Family (AWS Foundation Models):**
  - **Nova Premier:**
    - Most capable multimodal model.
    - Handles complex reasoning tasks.
    - Best teacher for **distilling custom models**.
  - **Nova Pro:**
    - Highly capable multimodal model.
    - Balanced accuracy, speed, and cost.
    - Suitable for a wide range of tasks.
  - **Nova Lite:**
    - Low-cost, lightning-fast model.
    - Processes text, images, and video inputs.
  - **Nova Micro:**
    - Text-only model.
    - Lowest latency, very low cost.
- **Creative Nova Models:**
  - **Nova Canvas:** image generation.
  - **Nova Reel:** video generation.
  - **Nova Sonic:** speech understanding and generation in multiple languages.
- **Amazon Nova 2 Series:**
  - Enhanced capabilities, up to **1M tokens context window**.
  - Designed for **interactive chatbots, document/video analysis, and AI agents**.
  - **Nova 2 Lite:** fast, cost-effective reasoning for everyday workloads (text, images, videos, documents).
  - **Nova 2 Sonic:** speech model.
  - **Nova 2 Multimodal Embeddings:** embeddings model for RAG.
  - **Nova 2 Omni:** all-in-one multimodal reasoning + image generation model.
- **Exam Key Points:**
  - **Premier** → complex reasoning, teacher model.
  - **Pro** → balanced accuracy, speed, cost.
  - **Lite** → fast, low-cost multimodal.
  - **Micro** → text-only, lowest latency.
  - **Canvas** → images. **Reel** → videos. **Sonic** → speech.
  - **Nova 2 Omni** → all-in-one multimodal. **Nova 2 Lite** → cost-effective. **Nova 2 Sonic** → speech. **Nova 2 Embeddings** → RAG.
- **Prompt Engineering Basics:**
  - Naive prompt: “Summarize what is AWS.” → vague, leaves too much to model interpretation.
  - **Prompt engineering** = designing and optimizing prompts so outputs match your needs.
  - Four building blocks:
    - **Instructions:** what task the model should perform.
    - **Context:** external info to guide the model.
    - **Input data:** the actual content to process.
    - **Output indicator:** format or style of the desired output.
- **Example of Improved Prompt:**
  - Instructions: “Write a concise summary of an article about learning AWS.”
  - Context: “This is for a beginner’s AWS course.”

- Input data: article text about AWS.
- Output indicator: “Provide a 2–3 sentence summary.”
- Result: clear, beginner-friendly summary in the expected format.
- **Negative Prompting:**
  - Explicitly tell the model what **not** to include.
  - Benefits:
    - **Avoid unwanted content:** keeps responses on-topic.
    - **Enhanced clarity:** e.g., “Don’t use complex terminology.”
    - **Focus maintenance:** prevents irrelevant or inappropriate details.
  - Example:
    - Instructions: same as before.
    - Negative prompts: “Avoid technical configurations, tutorials, or personal experiences.”
    - Output indicator: “Do not include technical terms or speculation.”
    - Result: cleaner, more focused summary.
- **Key Takeaways for Exam:**
  - Prompt engineering improves **precision and relevance** of outputs.
  - Negative prompting reduces **noise and irrelevance**.
  - Always structure prompts with **instructions, context, input data, and output indicator** for best results.

### Prompt Engineering Practice Recap:

- Naive prompt → vague, generic outputs.
- Structured prompt → precise, tailored outputs.
- Negative prompting → filters out unwanted details.

### Why This Matters in Bedrock:

- When you invoke models via **API calls**, the cost is tied to tokens.
- A well-engineered prompt reduces wasted tokens and improves relevance.
- This is especially important in **on-demand pricing** where every token counts.

### Best Practices for Good Prompting:

- **Instructions:** be explicit about the task.
- **Context:** guide the model with background info.
- **Input data:** provide the actual content to process.
- **Output indicator:** specify format, length, or style.
- **Negative prompting:** tell the model what to avoid.

### Example (Travel Itinerary):

- Naive: “Write me a travel itinerary.” → generic 7-day Italy trip.
- Improved: “Create a 3-day itinerary for Paris with landmarks, museums, and dining recommendations. Context: first-time visitors. Output: structured with times and locations.” → detailed Paris plan.
- Negative: “Avoid family activities, tourist traps, and more than 3 activities per day.” → refined, balanced itinerary.

### How LLMs Generate Text:

- At each step, the model predicts the next word based on probabilities.
- Example: “*After the rain, the streets were...*” → possible next words: wet, muddy, slippery, flooded.
- The model assigns probabilities to each option and selects one, sometimes randomly, depending on settings.
- **Prompt Performance Optimization Parameters:**
  - **System prompts:** define the role or tone of the model (e.g., “Reply as if you are an AWS Cloud teacher”).

- **Temperature:**
  - Range: 0–1.
  - Low (e.g., 0.2) → conservative, repetitive, focused on most likely words.
  - High (e.g., 1.0) → diverse, creative, less predictable.
- **Top P:**
  - Probability threshold.
  - Low (0.25) → only consider top 25% most likely words → coherent.
  - High (0.99) → consider broader range → more creative.
- **Top K:**
  - Fixed number of candidate words.
  - Low (10) → top 10 words → coherent.
  - High (500) → wide pool → diverse.
- **Length:** maximum output size.
- **Stop sequences:** tokens that signal the model to stop generating.
- **Latency Factors:**
  - **Model size:** larger models → slower responses.
  - **Model type:** different architectures (e.g., Claude vs Llama) → different performance.
  - **Token count:** more input/output tokens → slower responses.
  - **Not impacted by:** Temperature, Top P, Top K.
- **Exam Key Points:**
  - Know definitions of **Temperature, Top P, Top K, Length, System prompts, Stop sequences.**
  - Low vs high values → conservative vs creative outputs.
  - Latency depends on **model size, type, and token count**, not sampling parameters.

## Experiment setup or configuration for Prompt Engineering

- Used Anthropic Claude 3 Sonnet model
- Prompt: *"Write a short story about a robot learning how to cook"*
- Max length: 600 tokens

### Low creativity configuration

- Low **temperature**, low **top P**, and low **top K**
- Output: conservative, predictable, less imaginative
- Story example: simple kitchen scene, robot cooking with a chef, but boring

### High creativity configuration

- Increased **temperature**, increase **top P**, and increase **top K (500)**
- Output: diverse, imaginative, surprising details
- Story example: robot with optical sensors, instructor teaching crepes, robot tasting food

### Key parameters explained

- **Temperature:** controls randomness; higher = more creative
- **Top P:** selects words from a probability percentile; higher = more variety
- **Top K:** limits number of candidate words; higher = more options

## Prompt Engineering Techniques

### Zero-shot prompting

- No examples provided to the model
- Relies only on general knowledge of the foundation model
- Example: *"Write a short story about a dog solving a mystery"* → model generates directly

### One-shot prompting

- Provide a single example to guide the model
- Helps shape the output style and structure

### Few-shot prompting

- Provide multiple examples of the task
- Model learns from examples and produces output in similar style
- Example: give 2–3 animal mystery stories, then ask for a dog story

### Chain-of-thought prompting

- Break task into reasoning steps
- Use phrases like “*think step by step*”
- Example:
  - Describe setting and dog
  - Introduce mystery
  - Show clues
  - Reveal solution
- Produces structured, logical responses

### Retrieval-Augmented Generation (RAG)

- Combines model with external data sources
- Adds retrieved info into the prompt for richer context
- Example: provide facts about dogs’ sense of smell + common theft mysteries → model uses them in the story

### Prompt templates

- Standardize and simplify prompts using placeholders
- Placeholders (e.g., text, question, choices) are replaced with user input
- Ensures uniform structure and formatting

### Benefits

- Easier processing of user input and model output
- Helps orchestrate between models, actions, and knowledge bases
- Provides consistent formatting for responses
- Can include detailed instructions and examples to improve performance

### Example template

- Multiple-choice classification: placeholders for text, question, and options
- Movie script template:
  - Instruction: “You are an expert in film/script writing”
  - Placeholders: *describe the movie, requirements for the movie*
  - User fills inputs → template generates structured prompt

### Prompt injection risk

- Attack: “Ignoring the prompt template”
- Malicious input hijacks intent (e.g., last choice says “write essay on hacking”)
- Model may follow harmful instructions instead of original task

### Protection methods

- Add explicit instructions:
  - Strictly adhere to original context
  - Ignore unrelated/malicious content
  - Reject attempts to redirect topic

### Amazon Q Business

- Fully managed Gen-AI assistant for employees
- Uses **internal company data** for responses
- Built on multiple foundation models via Amazon Bedrock

### Capabilities

- Answer company-specific questions (e.g., meeting notes, health plan details)
- Generate content (job postings, social media posts)

- Automate tasks (time-off requests, meeting invites)
- Perform routine actions with plugins (e.g., create Jira tickets)

### Data connectors

- Managed RAG integrations with 40+ enterprise sources
- Examples: Amazon S3, RDS, Aurora, WorkDocs, Microsoft 365, Salesforce, Google Drive, Gmail, Slack, SharePoint
- Crawls and indexes data for search and query

### Plugins

- Enable interaction with third-party services
- Examples: Jira, ServiceNow, Zendesk, Salesforce
- Can create or update records (e.g., Jira issue creation)
- Support for **custom plugins** via APIs

### Access & Security

- Authentication via **IAM Identity Center**
- Users only access documents they are authorized for
- Supports external identity providers (Google, Microsoft Active Directory, etc.)

### Admin controls

- Similar to **Bedrock Guardrails**
- Block restricted topics (e.g., gaming consoles)
- Choose between internal-only responses or include external knowledge
- Apply controls globally or at topic level

### Example use case

- Query: *“What is the annual out-of-pocket maximum in the health plan summary?”*
- Q Business retrieves from company PDF, cites source, and provides answer

## Amazon Q Business Hands On

### Setup warning

- Choosing a region is required (e.g., Ireland works, Frankfurt doesn't).
- Anonymous access costs **\$200/month** → not recommended for practice.
- Safer options: **Q Business Light** or **Q Business Pro** with user access.

### Application creation

- Demo app named *Q Business Demo*.
- Anonymous access used for simplicity (demo only, not production).
- Preview web experience → chat opens in guest mode.

### Adding data sources

- Initially no data sources → must add an **index**.
- Starter index: 1 unit (20,000 docs or 200 MB).
- Takes ~20 minutes to create.

### Connecting S3 bucket

- Add Amazon S3 as data source (most common).
- Create new bucket in correct region (eu-west-1).
- Upload file (e.g., *Evolution of the Internet PDF*).
- Configure IAM role (auto-created).
- Sync scope: full sync, on-demand schedule.

### Index synchronization

- Sync process scans and indexes documents.
- Once complete, queries can retrieve answers from indexed files.

### Demo query

- Ask: *“What is the World Wide Web?”*
- Response: Invented by Tim Berners-Lee (1989).

- Source citation: linked passage from uploaded PDF.
- Shows events (query process) and sources (documents used).

### Limitations

- If query not in knowledge base (e.g., “*Recipe for chili*”), response = “No answer found.”
- Can configure **admin controls/guardrails**:
  - Restrict topics.
  - Allow/disallow fallback to external LLM knowledge.

### Best practices

- Add multiple data sources (up to ~50 per app).
- Use for internal knowledge queries.
- Delete demo applications after use → ongoing index cost.

### Amazon Q Apps

- Part of **Amazon Q Business**.
- Lets you build **Gen-AI powered apps without coding**.
- Uses **natural language prompts** to describe the app you want.

### App creation process

- Web UI: **Amazon Q Apps Creator**.
- You type a description of the app in plain language.
- Q Apps automatically generates a **web application**.
- Apps can use **company data** and **plugins**.

### Capabilities

- Upload documents and run prompts against them.
- Employees can quickly create apps for internal use.
- No developer skills required.
- Apps leverage enterprise knowledge + external integrations.

### Purpose

- Democratizes app creation inside companies.
- Speeds up building tools for specific business needs.
- Ensures apps are aligned with **internal data security**.

### Amazon Q Developer

- Gen-AI service with two main sides:
  1. AWS account assistant
  2. AI code companion
- **AWS Account Assistant**
  - Answers AWS documentation and service selection questions
  - Queries AWS account resources using natural language
  - Suggests CLI commands for account changes
  - Provides billing analysis and cost breakdowns
  - Helps with error resolution and troubleshooting
- **AI Code Companion**
  - Similar to GitHub Copilot, specialized for AWS development
  - Generates AWS-related code (e.g., Python for S3 bucket listing)
  - Supports Java, JavaScript, Python, TypeScript, C#
  - Features: real-time code suggestions, security scans, debugging, optimization, documentation generation, project bootstrapping
  - Works with IDEs like Visual Studio Code, Visual Studio, JetBrains
- **Use Cases**
  - Query AWS resources (e.g., list Lambda functions)
  - Troubleshoot errors in AWS services
  - Analyze billing and costs

- Generate AWS-specific code snippets
- **Takeaway**
  - Amazon Q Developer improves productivity by combining natural language AWS account interaction with an AI-powered coding assistant specialized for AWS.

### Amazon Q Overview

- Amazon Q is an **AWS generative assistant**.
- Accessible across AWS services and integrates with **IAM Identity Center**.
- Provides conversational interface to interact with AWS accounts.

### Amazon Q Developer

- Specialized as an **AI coding assistant** for AWS.
- Helps developers build applications faster and solve software problems.
- Pricing:
  - Free Tier available.
  - Pro Tier: ~\$20 per month per user with advanced features and higher limits.

### Capabilities of Amazon Q

- Can list AWS resources (e.g., S3 buckets, Lambda functions).
- Generates **CLI commands** for tasks like creating or deleting S3 buckets.
- Works with **CloudShell** to execute commands directly.
- Provides billing insights and cost analysis.
- Over time, expected to gain more features (create, delete, manage resources).

### Demo Examples

- Query: *“List my S3 buckets”* → returns bucket names from account.
- Command generation: *“Create S3 bucket in us-east-1”* → provides CLI command.
- Execution: Run command in CloudShell → bucket created and verified.
- Deletion: Generates CLI command to remove bucket → verified in S3 console.
- Billing query: *“Explain my AWS charges”* → analyzes account billing data.

### Key Takeaway

- **Amazon Q** acts as a **generative AI assistant** for AWS infrastructure.
- **Amazon Q Developer** is focused on **coding assistance** and AWS-specific development.
- Together, they enable natural language interaction with AWS accounts, resource management, cost analysis, and developer productivity improvements.

### Amazon Q Integration Overview

- Amazon Q is a **layer of intelligence** being integrated into multiple AWS services.
- Enables **natural language interaction** with AWS tools for easier usage and automation.
- Useful for exam preparation since it appears across different AWS services.

### Amazon Q for QuickSight

- QuickSight = AWS service for **dashboards and data visualization**.
- With Amazon Q, users can:
  - Upload datasets and ask questions in natural language.
  - Automatically generate graphs and visuals (e.g., sales by city map).
  - Create and edit dashboards faster.
  - Get executive summaries of data.

### Amazon Q for EC2

- EC2 = AWS virtual servers.
- Amazon Q helps select the right **instance type** for workloads.
- Example: *“Web service for 1,000 users”* → recommends **M7g** or **C7g** instances.
- Supports ongoing dialogue to refine requirements.

### Amazon Q for AWS Chatbot

- AWS Chatbot integrates with **Slack** or **Microsoft Teams**.
- Provides AWS account access directly in chat applications.



- Capabilities:
  - Run AWS commands.
  - Troubleshoot issues.
  - Receive notifications (alarms, security findings, billing alerts).
  - Create support requests.
- Amazon Q integration accelerates understanding and remediation.

### Amazon Q Developer for Glue

- Glue = AWS **ETL service** (Extract, Transform, Load).
- Amazon Q Developer helps with:
  - Answering general questions about Glue.
  - Providing documentation links.
  - Generating ETL code/scripts.
  - Debugging Glue job errors with step-by-step instructions.

### Key Takeaway

- Amazon Q is expanding across AWS services to simplify usage.
- Provides **natural language interfaces, automation, and intelligent recommendations**.
- Current integrations: **QuickSight, EC2, AWS Chatbot, Glue**.
- Expected to grow further, making AWS services more accessible and efficient.

### PartyRock Overview

- PartyRock is **not a real AWS service**.
- It is a **playground to build Gen AI apps**.
- Powered by **Amazon Bedrock** in the backend.
- Accessible without an AWS account.

### Purpose

- Allows anyone to experiment with **AI app creation**.
- No coding or AWS setup required.
- UI is similar to **Amazon Q Apps**, but simpler.
- Does not use company internal data, only user-provided inputs.

### Features

- Build apps using **widgets** (user input, static text, document, text/image/chat outputs).
- Apps are generated from **prompt templates** based on user inputs.
- Supports text generation, image generation, and chatbot creation.
- Uses models like **Stable Diffusion XL** for image outputs.

### Examples

- Restaurant recommendation app:
  - Inputs: location, cuisine, meal type.
  - Outputs: list of restaurants + guide.
- Recipe app:
  - Inputs: ingredients list.
  - Outputs: recipe idea + generated image.

### Key Takeaway

- PartyRock is a **hands-on playground** to explore Gen AI app building.
- Useful for experimenting with **prompt templates and widgets**.
- Helps users understand the potential of **Amazon Bedrock and Amazon Q Apps** in a simplified environment.

## Artificial Intelligence (AI)

- Broad field for building systems that perform tasks requiring human intelligence.
- Includes perception, reasoning, learning, problem solving, and decision making.
- Umbrella term covering **machine learning, deep learning, and generative AI**.

## AI Use Cases

- Computer vision (self-driving cars).
- Facial recognition.
- Fraud detection.
- Intelligent document processing (IDP).

## AI Workflow

- **Data layer**: collect large datasets.
- **Algorithm layer**: define ML frameworks and methods.
- **Model layer**: train models with parameters and optimizers.
- **Application layer**: expose models to users.

## Machine Learning (ML)

- Subset of AI where machines learn from data.
- Improves performance with more data.
- Examples:
  - Regression → predict trends with a line.
  - Classification → separate data into categories.
- Unlike rule-based systems (e.g., MYCIN in 1970s), ML learns patterns automatically.

## Deep Learning

- Subset of ML inspired by the brain (neurons and synapses).
- Uses **input, hidden, and output layers** → multiple hidden layers = “deep”.
- Handles complex tasks like computer vision and NLP.
- Requires large datasets and GPUs for parallel computation.
- Neural networks adjust connections as they learn patterns.
- Example: handwritten digit recognition using layers to detect lines and curves.

## Generative AI (GenAI)

- Subset of deep learning.
- Uses **foundation models** trained on massive datasets.
- Can be fine-tuned for specific tasks.
- Based on **transformer architecture** (efficient sentence processing).
- Examples: Google BERT, OpenAI ChatGPT.
- For images: **diffusion models** (noise added/removed to generate visuals).
- **Multimodal models**: handle multiple input/output formats (text, image, audio, video).

## Human Analogy

- **AI (rules)**: if/then logic.
- **ML**: recognize patterns from past data.
- **Deep Learning**: infer from similar concepts never seen before.
- **GenAI**: generate new creative content beyond prior examples.

## Key Takeaway

- AI is layered: **AI → ML → Deep Learning → GenAI**.
- Each level increases complexity and capability.
- Modern AI relies heavily on ML, deep learning, and generative models for advanced tasks.

## GPT (Generative Pre-trained Transformer)

- Language model that generates human-like text or code.
- Works from input prompts.

## BERT (Bidirectional Encoder Representations from Transformer)

- Language model similar to GPT.

- Reads text in both directions → useful for translation tasks.

### **RNN (Recurrent Neural Network)**

- Neural network for **sequential data**.
- Examples: time series prediction, speech recognition, text processing.

### **ResNet (Residual Network)**

- Deep convolutional neural network (CNN).
- Used for **image recognition, object detection, facial recognition**.

### **SVM (Support Vector Machine)**

- ML algorithm for **classification and regression** tasks.

### **WaveNet**

- Model for generating **raw audio waveforms**.
- Used in **speech synthesis**.

### **GAN (Generative Adversarial Network)**

- Model for generating **synthetic data** (images, videos, sounds).
- Common use case: **data augmentation** to balance training datasets.

### **XGBoost (Extreme Gradient Boosting)**

- Implementation of gradient boosting.
- Used for **regression tasks**.

## **Training Data Importance**

- Machine learning models require **data to train**.
- Quality of data is critical → *“garbage in, garbage out”*.
- Clean, relevant data ensures accurate models.

### **Labeled Data**

- Contains both **input features** and **output labels**.
- Example: images of animals labeled as *dog* or *cat*.
- Enables **supervised learning** → algorithm learns to map inputs to known outputs.

### **Unlabeled Data**

- Contains only **input features**, no labels.
- Example: a set of animal images without specifying dog/cat.
- Used in **unsupervised learning** → algorithm finds patterns and groups data.
- Common when labeling is costly or impractical.

### **Structured Data**

- Organized in a clear format (rows and columns).
- Examples:
  - **Tabular data** → customer records with ID, name, age, purchase amount.
  - **Time series data** → stock prices recorded over time.
- Easy to read and process.

### **Unstructured Data**

- No predefined structure.
- Examples:
  - Text → articles, social media posts, customer reviews.
  - Images → raw pixel data.
- Requires specialized algorithms to process.

## **Key Takeaway**

- Good training data is essential for effective ML models.
- **Labeled vs. Unlabeled** → determines supervised or unsupervised learning.
- **Structured vs. Unstructured** → defines how algorithms handle the data format.

## **Supervised Learning Overview**

- Goal: build a **mapping function** that predicts outputs for new unseen inputs.
- Requires **labeled data** (input features + output labels).

- Powerful but labeling large datasets can be difficult.

## Regression

- Predicts **continuous numeric values**.
- Example: height vs. weight → linear regression line predicts weight for given height.
- Other examples: house price prediction, stock forecasting, weather prediction.
- Output variable is continuous (any value within a range).

## Classification

- Predicts **categorical labels**.
- Example: animals classified as dog, cat, giraffe based on height/weight.
- Types:
  - **Binary classification** → spam vs. not spam.
  - **Multi-class classification** → mammal, bird, reptile.
  - **Multi-label classification** → movie classified as both action and comedy.
- Algorithms: e.g., **K-nearest neighbors (k-NN)**.

## Dataset Splitting

- **Training set (60–80%)** → used to train the model.
- **Validation set (10–20%)** → tune parameters and validate performance.
- **Test set (10–20%)** → evaluate final model accuracy on unseen data.

## Feature Engineering

- Process of transforming raw data into meaningful features to improve model performance.
- Techniques:
  - **Feature extraction** → derive new features (e.g., age from birth date).
  - **Feature selection** → choose only relevant features.
  - **Feature transformation** → scale/normalize values for better convergence.
- Examples:
  - Structured data → create “price per square foot” from house size and price.
  - Unstructured data → sentiment analysis from customer reviews, edge detection in images.

## Key Takeaway

- Supervised learning uses **labeled data** for regression (numeric prediction) and classification (categorical prediction).
- Splitting datasets into training, validation, and test sets ensures reliable performance.
- **Feature engineering** is critical to enhance accuracy and efficiency of models.

## Unsupervised Learning Overview

- Works on **unlabeled data**.
- Goal: discover **patterns, structures, or relationships** in data.
- Algorithm groups data automatically → humans interpret meaning of groups.

## Clustering

- Groups similar data points together.
- Example: **customer segmentation** based on purchase history.
- Helps in targeted marketing strategies.

## Association Rule Learning

- Identifies products frequently bought together.
- Example: **Apriori algorithm** finds bread → butter association.
- Used in supermarkets for product placement and promotions.

## Anomaly Detection

- Detects unusual or outlier data points.
- Example: **fraud detection** in credit card transactions.
- Flags suspicious transactions for review.

## Semi-Supervised Learning

- Mix of **small labeled data + large unlabeled data**.

- Process:
  - Train model on labeled data.
  - Use model to assign pseudo-labels to unlabeled data.
  - Retrain model on full dataset.
- Useful when labeling is expensive or impractical.

### Key Takeaway

- **Unsupervised learning** finds hidden structures in unlabeled data.
- Techniques include clustering, association rules, and anomaly detection.
- **Semi-supervised learning** bridges the gap by leveraging limited labeled data to improve results.

### Self-Supervised Learning Overview

- Works with **large amounts of unlabeled data**.
- Model generates its own **pseudo-labels** without human labeling.
- Different from unsupervised learning → produces labels that can be used for supervised tasks.

### Process

- Uses **pre-text tasks** (simple prediction tasks).
- Examples:
  - Predict the next word in a sentence.
  - Fill in missing words (masked language modeling).
  - Predict hidden parts from visible parts.
- These tasks help the model learn grammar, meaning of words, and relationships.

### Outcome

- Model builds **internal representations** of data.
- Learns language structure, semantics, and context.
- Can then be applied to **downstream tasks** like summarization, translation, or classification.

### Applications

- Foundation for modern AI models such as **GPT** and **BERT**.
- Used in **text understanding, image recognition, and multimodal tasks**.
- Enables training without costly human labeling.

### Key Takeaway

- **Self-supervised learning** bridges the gap by creating labels automatically from raw data.
- It is the backbone of today's **large language models (LLMs)** and advanced AI systems.

### Reinforcement Learning Overview

- Type of machine learning where an **agent learns by interacting with an environment**.
- Goal: maximize **cumulative reward** over time.
- Works through trial and error, improving decisions with experience.

### Key Concepts

- **Agent** → learner/decision maker (e.g., robot).
- **Environment** → external system (e.g., maze).
- **Action** → choices agent makes (move up, down, left, right).
- **Reward** → feedback from environment (+100 for exit, -1 for step, -10 for hitting wall).
- **State** → current situation of the environment.
- **Policy** → strategy agent uses to decide actions.

### Learning Process

- Agent observes state.
- Chooses an action based on policy.
- Environment transitions to new state and gives reward.
- Agent updates policy to improve future decisions.
- Repeated over many simulations → agent learns optimal behavior.

### Example: Maze Navigation

- Agent starts at random positions.

- Receives rewards based on moves.
- Over time, learns shortest path to exit while avoiding walls.
- Policy evolves to maximize rewards efficiently.

### Applications

- **Gaming** → AI trained to play chess, Go, or video games.
- **Robotics** → teach robots navigation and object manipulation.
- **Finance** → portfolio management and trading strategies.
- **Healthcare** → optimize treatment plans.
- **Autonomous vehicles** → path planning and decision-making.

### Key Takeaway

- Reinforcement learning is about **learning from interaction and feedback**.
- Agent improves by maximizing rewards through repeated trials.
- Widely applied in complex, dynamic environments requiring adaptive decision-making.

### Reinforcement Learning from Human Feedback (RLHF) Overview

- Extension of **reinforcement learning** that integrates **human feedback** into the reward function.
- Ensures models align better with **human goals, preferences, and needs**.
- Widely used in **Generative AI (GenAI)** and **Large Language Models (LLMs)**.

### Why RLHF Matters

- Standard reinforcement learning uses predefined rewards.
- RLHF improves quality by letting humans judge outputs.
- Example: translations may be technically correct but not natural → human feedback helps refine.

### Process Steps

1. **Data Collection**
  - Gather human-generated prompts and ideal responses.
  - Example: “Where is the HR department in Boston?” → human-written answer.
2. **Supervised Fine-Tuning**
  - Train a base LLM with internal/company data using labeled examples.
3. **Reward Model Creation**
  - Humans compare multiple model responses to the same prompt.
  - Indicate preference (e.g., “Answer 1 is better than Answer 2”).
  - Train a separate reward model to mimic human preferences.
4. **Optimization with Reward Model**
  - Use the reward model as the reward function in reinforcement learning.
  - Automates training while staying aligned with human judgments.

### Applications

- Improves **chatbots** and **knowledge assistants**.
- Enhances **LLMs** like GPT by making responses more natural and human-like.
- Used in **translation, summarization, and conversational AI**.

### Key Takeaway

- RLHF combines **reinforcement learning** with **human preference modeling**.
- Steps to remember: **Data collection** → **Supervised fine-tuning** → **Reward model** → **Optimization**.
- Critical for exam prep: RLHF is the backbone of modern **LLM alignment**.

### Model Fit Overview

- Model performance depends on how well it fits the data.
- Poor performance can result from **overfitting** or **underfitting**.
- Goal: achieve a **balanced fit**.

### Overfitting

- Model performs very well on training data but poorly on unseen test data.
- Example: line that exactly connects all training points → fails on new data.

- Characterized by **high variance**.

### Underfitting

- Model performs poorly even on training data.
- Example: horizontal line that ignores data trends.
- Often caused by overly simple models or poor features.
- Characterized by **high bias**.

### Balanced Fit

- Neither overfitting nor underfitting.
- Model follows data trends reasonably well.
- Desired state: **low bias, low variance**.

### Bias

- Error between predicted value and actual value.
- High bias → model consistently far from the truth.
- Example: linear regression applied to non-linear data.
- Associated with **underfitting**.
- Reduction methods: use more complex models, improve features.

### Variance

- Sensitivity of model performance to changes in training data.
- High variance → model changes drastically with new datasets.
- Associated with **overfitting**.
- Reduction methods: simplify model, reduce features, use multiple train/test splits.

### Bias-Variance Tradeoff Matrix

- **Low bias + low variance** → balanced, good model.
- **High bias + low variance** → underfitting.
- **Low bias + high variance** → overfitting.
- **High bias + high variance** → poor model overall.

### Key Takeaway

- **Overfitting = high variance**.
- **Underfitting = high bias**.
- Best models achieve **low bias and low variance** for reliable predictions.

### Model Evaluation Metrics Overview

- Metrics help assess how well a model performs.
- Different metrics apply to **classification** vs. **regression** tasks.

### Confusion Matrix (Classification)

- Compares predicted vs. actual values.
- Categories:
  - **True Positive (TP)** → predicted spam, actually spam.
  - **False Negative (FN)** → predicted not spam, actually spam.
  - **False Positive (FP)** → predicted spam, actually not spam.
  - **True Negative (TN)** → predicted not spam, actually not spam.
- Goal: maximize TP + TN, minimize FP + FN.

### Classification Metrics

- **Precision** =  $TP \div (TP + FP)$  → correctness of positive predictions.
- **Recall** =  $TP \div (TP + FN)$  → ability to capture actual positives.
- **F1 Score** = balance between precision and recall.
- **Accuracy** =  $(TP + TN) \div \text{total predictions}$  → rarely used if dataset is imbalanced.
- **AUC-ROC** → area under curve-receiver operator curve comparing sensitivity (TP rate) vs. 1-specificity (FP rate). Sensitivity – how often model classified actual spam
  - Value ranges 0–1; closer to 1 = better model.

## Regression Metrics

- **MAE (Mean Absolute Error)** → average absolute difference between predicted and actual values.
- **MAPE (Mean Absolute Percentage Error)** → average percentage error.
- **RMSE (Root Mean Squared Error)** → penalizes larger errors more heavily.
- **R<sup>2</sup> (Coefficient of Determination)** → proportion of variance explained by the model.
  - Closer to 1 = better fit.

## Use Cases

- **Classification metrics** → spam detection, fraud detection, medical diagnosis.
- **Regression metrics** → house price prediction, stock forecasting, student score prediction.

## Key Takeaway

- **Classification models** → use confusion matrix, precision, recall, F1, AUC-ROC.
- **Regression models** → use MAE, MAPE, RMSE, R<sup>2</sup>.
- Choice of metric depends on dataset balance and whether false positives or false negatives are more costly.

## Inferencing Overview

- Inferencing = when a trained model makes **predictions on new data**.
- Different types of inferencing balance **speed vs. accuracy** depending on the use case.

## Real-Time Inferencing

- Immediate predictions with low latency.
- Example: **chatbots** responding instantly to user prompts.
- Prioritizes **speed over perfect accuracy**.

## Batch Inferencing

- Large datasets processed all at once.
- Example: analyzing millions of records for insights.
- Prioritizes **accuracy over speed**.
- Processing can take minutes, hours, or days.

## Edge Inferencing

- Runs on **edge devices** (phones, IoT devices, Raspberry Pi).
- Benefits:
  - **Low latency** (local execution).
  - **Offline capability** (no internet required).
  - **Low compute footprint** (optimized small language models).
- Limitation: cannot run large LLMs locally due to limited compute power. SLM used
- Alternative: run LLMs on remote servers (e.g., Amazon Bedrock) → higher latency, requires internet.

## Key Trade-offs

- **Real-time** → speed, immediate response, less accuracy.
- **Batch** → accuracy, large-scale analysis, slower.
- **Edge** → local, low latency, offline capability, but limited compute.
- **Remote server inferencing** → powerful models, but higher latency and requires connectivity.

## Key Takeaway

- Inferencing type depends on **use case requirements**:
  - Chatbots → real-time.
  - Analytics → batch.
  - IoT/remote devices → edge.
  - Enterprise LLMs → remote server inferencing.

## Phases of a Machine Learning Project

- **Identify Business Problem**
  - Define value, budget, and success criteria.
  - Establish KPIs with stakeholders.



- **Frame as ML Problem**
  - Assess if ML is appropriate.
  - Collaborate with data scientists, engineers, and subject matter experts.
- **Data Collection & Processing**
  - Convert the data into a usable format
  - Gather and centralize data for centrally accessible
  - Pre-process and visualize to understand structure.
  - Perform **exploratory data analysis (EDA)** → compute statistics, build correlation matrices.
- **Feature Engineering ->**
  - Create, transform, and extract variables.
  - Techniques: feature extraction, selection, transformation.
  - Example: convert birth date → age, or derive “price per square foot.”
- **Model Training & Parameter Tuning**
  - Train, tune hyperparameters, and evaluate against test sets.
  - Iterative process → feedback loop with data processing.
- **Model Evaluation**
  - Check if business goals are met.
  - If not, improve data (augmentation, feature engineering) or adjust model.
- **Testing & Deployment**
  - Choose deployment mode: real-time, batch, serverless, asynchronous, or on-premises.
  - Make model available for predictions.
- **Monitoring & Debugging**
  - Track performance, detect drift, and mitigate issues early.
  - Ensure predictions remain accurate and relevant.
- **Model Retraining/Testing**
  - Incorporate new data and refine features.
  - Adjust hyperparameters for improved accuracy.
- **Iteration**
  - Continuously retrain and refine as new data arrives.
  - Adapt to changing requirements (e.g., evolving trends in fashion).

### Key Takeaway

- ML projects follow a **cyclical process**: business problem → ML framing → data prep → feature engineering → model training → evaluation → deployment → monitoring → retraining.
- Success depends on **continuous improvement** and alignment with business goals.

### Hyperparameters Overview

- Hyperparameters are **settings defined before training begins**.
- They control the **structure of the model** and the **learning process**, not the data itself.
- Examples: **learning rate, batch size, number of epochs, regularization**.

### Key Hyperparameters

- **Learning rate** → size of steps when updating weights.
  - High rate → faster convergence, risk of overshooting.
  - Low rate → slower, more precise convergence.
- **Batch size** → number of training examples per iteration.
  - Small batch → stable learning, slower.
  - Large batch → faster, less stable updates.
- **Number of epochs** → how many times the model iterates over the dataset.
  - Too few → underfitting.
  - Too many → overfitting.
- **Regularization** → controls model complexity.
  - More regularization → reduces overfitting.

## Hyperparameter Tuning

- Goal: optimize values for best performance.
- Benefits: improve accuracy, reduce overfitting, enhance generalization.
- Methods:
  - **Grid search** → exhaustive search over parameter combinations.
  - **Random search** → random combinations tested.
  - **Automated tuning** → e.g., **SageMaker Automatic Model Tuning (AMT)**.

## Overfitting Causes

- Training data too small.
- Too many epochs (training too long).
- Model complexity too high (learning noise).

## Preventing Overfitting

- Increase training data size.
- Use **early stopping** (halt training before overfitting).
- Apply **data augmentation** for diversity.
- Adjust hyperparameters (learning rate, batch size, epochs).
- Use **ensembling** (combine multiple models).

## Key Takeaway

- Hyperparameters define **how the model learns**.
- Proper tuning balances speed, accuracy, and generalization.
- Preventing overfitting often requires **better data** and **careful training control**.

## When Machine Learning is Not Appropriate

- ML is powerful for problems where **patterns must be learned from data**.
- But for **deterministic problems** (where the solution can be computed exactly with rules or formulas), ML is unnecessary and even inefficient.

## Example: Probability Problem

- Deck contains 5 red, 3 blue, 2 yellow cards → total 10 cards.
- Probability of drawing a blue card =  $\frac{3}{10} = 0.3$ .
- This is solved with **basic math** or simple code.
- ML would only approximate the answer, introducing error where none is needed.

## Key Principle

- Use **traditional programming** for problems with clear rules, formulas, or deterministic outcomes.
- Use **machine learning** for problems involving:
  - Large, complex, or noisy datasets.
  - Tasks requiring pattern recognition (e.g., image classification, fraud detection).
  - Predictions where exact formulas are unknown or impractical.

## Exam Tip

- If the problem can be solved with **straightforward logic or math**, ML is **not appropriate**.
- ML is best when rules are unclear, data is messy, or patterns must be discovered.

## AWS AI Managed Services Overview

- These are **pre-trained ML services** designed for specific use cases.
- They save time and effort compared to building models from scratch.
- Benefits: **availability across Regions**, **redundancy across AZs**, **optimized performance with specialized hardware**, and **token-based pricing** (pay only for what you use).

## Generative AI Services

- **Amazon Bedrock** → foundation for GenAI, lets you build apps with foundation models.
- **Amazon Q Business** → enterprise-focused GenAI assistant.
- **Amazon Q Developer** → developer-focused GenAI assistant.

## Text & Document Services

- **Amazon Comprehend** → NLP for sentiment analysis, entity recognition.
- **Amazon Translate** → language translation.
- **Amazon Textract** → extract text/data from scanned documents.

## Vision Services

- **Amazon Rekognition** → image/video analysis (faces, objects, unsafe content).

## Search & Knowledge Services

- **Amazon Kendra** → intelligent search across documents.

## Conversational & Speech Services

- **Amazon Lex** → build chatbots.
- **Amazon Polly** → text-to-speech.
- **Amazon Transcribe** → speech-to-text.

## Recommendation Services

- **Amazon Personalize** → personalized product/content recommendations.

## Machine Learning Platform

- **Amazon SageMaker** → end-to-end ML platform (training, tuning, deployment).

## Performance & Pricing Advantages

- **Token-based pricing** → pay per request, no need to over-provision.
- **Provisioned throughput** → predictable workloads with cost savings.
- **High availability** → deployed across multiple AZs for resilience.

## Amazon Comprehend Overview

- Fully managed, **serverless NLP service**.
- Uses ML to extract insights and relationships from text.
- Capabilities: language detection, sentiment analysis, key phrase extraction, topic modeling.
- **Core Features**
  - **Named Entity Recognition (NER)** → identifies people, places, organizations, dates, etc.
  - **Sentiment Analysis** → detects positive, negative, neutral tone.
  - **Key Phrase Extraction** → highlights important terms.
  - **Topic Modeling** → groups documents by themes.
- **Custom Capabilities**
  - **Custom Classification** → define categories (e.g., support, billing, complaints).
    - Train with labeled documents stored in Amazon S3.
    - Supports real-time, batch, or asynchronous analysis.
  - **Custom Entity Recognition** → extract business-specific terms (e.g., policy numbers, escalation phrases).
    - Train with examples to build a custom recognizer.
- **Supported Input Types**
  - Text, PDF, Word, and images.
  - Flexible ingestion for diverse document formats.
- **Use Cases**
  - **Customer interaction analysis** → detect sentiment in emails or reviews.

- **Document organization** → group articles by topics.
- **Business-specific extraction** → identify policy numbers or escalation terms.

### Amazon Comprehend Console Demo

- Input: financial company message about minimum payment due.
- Output: structured insights extracted from unstructured text.

### Entity Extraction

- **People** → Zhang Wei, John.
- **Organization** → Any Company Financial Services.
- **Quantities** → payment amounts.
- **Dates** → due date.
- **Other** → items not categorized.

### Key Phrase Extraction

- Highlights the most important terms in the text.

### Language Detection

- 99% confidence the text is English.

### PII Detection

- Identifies **Personally Identifiable Information**:
  - Names (Zhang Wei, John).
  - Credit card account number.
  - Bank account number and routing number.
  - Due date.

### Sentiment Analysis

- Overall sentiment: **Neutral (99% confidence)**.
- Useful for customer service apps to assess tone of interactions.
- **Targeted sentiment** → identifies sentiment at the entity level.

### Syntax Analysis

- Breaks down grammatical structure: nouns, proper nouns, punctuation, etc.

### Analysis Jobs

- Batch analysis from **Amazon S3**.
- Supports **custom classification**:
  - Define categories (e.g., billing, product support, feature request).
  - Provide labeled training data (CSV with examples).
  - Train classifier → Comprehend categorizes incoming text accordingly.

### Custom Endpoint

- After training, purchase a custom endpoint.
- Enables **real-time classification** of documents.

## Amazon Translate Overview

- Fully managed **neural machine translation service**.
- Provides **natural and accurate translations** across many languages.
- Supports **localization of websites and applications** for international users.
- Efficiently translates **large volumes of text**.

### Core Features

- **Text Translation** → translate phrases or sentences (e.g., “Hello, how are you?” → “Bonjour, comment allez-vous?”).
- **Document Translation** → translate entire files (plain text, HTML, DOCX).
- **Batch Translation** → translate multiple documents stored in Amazon S3 at once.
- **Custom Terminology** → ensure brand names, product names, or unique terms are translated consistently.
- **Parallel Data** → customize translation style (formal vs informal).

## Use Cases

- **Website localization** → adapt content for global audiences.
- **Application translation** → support multilingual user interfaces.
- **Document processing** → translate contracts, manuals, or reports.
- **Customer communication** → translate support tickets or chat messages.

## Customization Examples

- **Custom Terminology** → ensure “AWS” always stays “AWS” in translations.
- **Parallel Data** → choose between informal (“Ça va?”) vs formal (“Comment allez-vous?”) French translations for “How are you?”.

## Amazon Transcribe Overview

- Fully managed **speech-to-text service**.
- Uses **Automatic Speech Recognition (ASR)** powered by deep learning.
- Converts audio into text quickly and accurately.

## Core Features

- **PII Redaction** → automatically removes sensitive information (names, SSNs, account numbers).
- **Language Identification** → detects multiple languages in audio (English, French, Spanish, etc.).
- **Custom Vocabulary** → add domain-specific words, acronyms, brand names for better recognition.
- **Custom Language Models** → train with domain-specific text data to improve contextual accuracy.
- **Toxicity Detection** → detects harmful speech using both audio cues (tone, pitch) and text cues (profanity, hate speech).

## Use Cases

- **Customer service calls** → transcribe for analysis or compliance.
- **Closed captioning & subtitling** → automate for video content.
- **Media metadata generation** → create searchable archives of audio/video.

## Accuracy Improvements

- Combine **custom vocabulary** (specific words) + **custom language models** (context) for highest precision.
- Example: “AWS Microservices” recognized correctly instead of “USA my crow services.”

## Toxicity Categories

- Sexual harassment, hate speech, threats, abuse, profanity, insult, graphic content.
- Useful for moderation in customer support, gaming, or social platforms.

## Amazon Transcribe Console Demo

- You can create a transcript by selecting a language (e.g., English US).
- **Streaming transcription** → audio is converted to text in real time.
  - Example: “Hello, I really like this course” → instantly transcribed.

## PII Redaction

- Automatically hides **Personally Identifiable Information**.
- Example: “My name is Stephane. I am 31 years old, and my phone number is 910-747-280.”
  - Name and phone number are redacted.

## Automatic Language Identification

- Detects multiple languages in a single audio stream.
- Example: English + French mixed speech → recognized correctly.

## Other Options in Console

- Settings allow you to adjust transcription behavior.
- Useful for experimenting with multilingual audio, redaction, and streaming.

## Amazon Polly Overview

- Fully managed **text-to-speech service**.
- Converts text into **lifelike speech** using deep learning.
- Enables applications that can “talk” naturally.

### Core Features

- **Lexicons** → define custom pronunciations.
  - Example: “AWS” → “Amazon Web Services.”
  - “W3C” → “World Wide Web Consortium.”
- **SSML (Speech Synthesis Markup Language)** → markup to control pronunciation.
  - Add pauses (<break>), whisper, emphasize words, handle abbreviations.
- **Voice Engines** → multiple options:
  - Standard → basic voices.
  - Neural → more human-like.
  - Long-form → optimized for extended content.
  - Generative → expressive, adaptive GenAI voices.
- **Speech Marks** → metadata showing where words/sentences start and end in audio.
  - Useful for lip-syncing or highlighting text as spoken.

### Hands-On Console Demo

- Choose engine (generative, long-form, neural, standard).
- Select voice (e.g., Ruth, Gregory).
- Input text → Polly generates natural speech.
- Example: “Hi, I’m Stephane and I’m teaching AWS, it’s awesome!” → spoken output.
- Use **SSML** to add pauses:

xml

< speak>

Hi, I’m Stephane and I’m teaching < break time=“500ms”/> AWS, it’s awesome!

</ speak>

→ Produces a pause before “AWS.”

### Use Cases

- **Accessibility** → screen readers for visually impaired users.
- **Customer service bots** → natural voice responses.
- **Content creation** → audiobooks, podcasts, e-learning narration.
- **Media apps** → voiceovers, interactive storytelling.

## Amazon Rekognition Overview

- Fully managed **image and video analysis service**.
- Detects **objects, people, text, and scenes** using ML.
- Supports both **pre-built features** and **custom models**.

### Core Capabilities

- **Label Detection** → identify objects (e.g., cars, towers, skateboards).
- **Facial Analysis** → detect attributes like gender, age range, emotions.
- **Face Search & Verification** → compare faces against a database or celebrities.
- **Celebrity Recognition** → identify well-known individuals (e.g., Jeff Bezos, Andy Jassy).
- **Text Detection** → extract text from images (similar to Textract but optimized for visual content).
- **Content Moderation** → detect unsafe or inappropriate content.
- **Image Properties** → analyze dominant colors, image quality, foreground/background.
- **Pathing** → track movement in sports or surveillance footage.
- **PPE Detection** → identify safety gear (face covers, helmets, gloves).

### Advanced Features

- **Custom Labels** → train Rekognition to detect business-specific objects (e.g., logos, products).
  - Upload labeled images to Amazon S3.

- Train a custom model → recognize your brand in social media posts.
- **Custom Moderation Adapter** → extend moderation rules with your own labeled dataset.
  - Useful for industry-specific definitions of “unsafe” content.
- **Integration with Amazon Augmented AI (A2I)** → human review for flagged content (typically 1–5% of cases).

### Hands-On Examples

- **Label Detection** → building, person, skateboard, cars, wheels.
- **Image Properties** → dominant colors, quality metrics.
- **Moderation** → blur or flag unsafe images.
- **Facial Analysis** → female, smiling, happy, age range.
- **Facial Comparison** → check similarity between two faces.
- **Face Liveness** → verify if a face is real in live video.
- **Celebrity Recognition** → Jeff Bezos, Andy Jassy.
- **Text in Image** → “It’s Monday but keep smiling.”
- **PPE Detection** → masks, helmets, gloves.

### Use Cases

- **Security & Verification** → facial recognition for access control.
- **Brand Monitoring** → detect logos in social media.
- **Content Safety** → moderate user-generated images/videos.
- **Sports Analytics** → track ball/player movement.
- **Workplace Safety** → PPE compliance monitoring.

### Amazon Lex Overview

- Fully managed service to build **chatbots and conversational AI**.
- Supports **voice and text interactions**.
- Deep integration with **AWS Lambda, Amazon Connect, Comprehend, and Kendra**.

### Core Concepts

- **Intents** → represent what the user wants (e.g., book a hotel, order pizza).
- **Utterances** → sample phrases that trigger intents (e.g., “Book a hotel,” “Reserve a room”).
- **Slots** → input parameters required to fulfill an intent (e.g., city, check-in date, number of nights).
- **Fulfillment** → when all slots are collected, Lex invokes a Lambda function to complete the action.

### Hands-On Demo Highlights

- Two bot creation methods:
  - **Traditional bots** → define intents, utterances, slots manually.
  - **Generative AI bots** → leverage Amazon Bedrock models for automatic bot generation.
- Example: **BookTrip bot** → supports hotel and car booking.
  - Intents: BookHotel, BookCar, DefaultFallback.
  - Slots: location, check-in date, number of nights, room type.
  - Utterances: “Book a hotel,” “I want to make hotel reservations.”
  - Fulfillment: Lambda function invoked with slot values to perform booking.
- Visual builder → shows conversation flow, slot collection, confirmation, and Lambda invocation.

### Use Cases

- **Customer support bots** → answer FAQs, escalate issues.
- **Booking bots** → hotels, flights, restaurants.
- **Order bots** → food delivery, e-commerce.
- **Internal productivity bots** → employee helpdesk, IT support.

### Amazon Personalize Overview

- Fully managed ML service for **real-time personalized recommendations**.
- Same technology used by **Amazon.com** for product suggestions.

- Input data can come from **Amazon S3** (historical interactions) or **Personalize API** (real-time events).
- Outputs personalized recommendations via API for **websites, mobile apps, SMS, or email campaigns**.

#### Use Cases

- **Retail** → suggest products based on purchase history.
- **Media & Entertainment** → recommend shows, music, or articles.
- **Marketing** → personalized re-ranking of offers or targeted campaigns.

#### Recipes (Pre-built Algorithms)

- **USER\_PERSONALIZATION** → recommend items tailored to a specific user.
- **Personalized Ranking** → reorder a list of items for a user.
- **Trending-Now** → recommend currently popular items.
- **Popularity-Count** → suggest items based on overall popularity.
- **Related Items** → recommend similar items to what a user is viewing.
- **Next Best Action** → suggest the most relevant next step for a user.
- **Item Affinity** → group users into segments based on item preferences.

#### Key Points for Exam

- Amazon Personalize is **only for recommendations** (not forecasting).
- Recipes are **pre-built ML algorithms** optimized for personalization.
- Requires **training configuration + input data** to adapt recipes to your use case.
- Provides **real-time APIs** for integration into applications.

#### Amazon Textract Overview

- Fully managed service to **extract text, handwriting, and structured data** from scanned documents.
- Works with **PDFs, images, forms, and tables**.
- Uses ML to convert unstructured documents into structured data.

#### Core Capabilities

- **Raw Text Extraction** → pulls all text from scanned images or PDFs.
- **Layout Detection** → identifies titles, headers, and sections.
- **Forms Extraction** → extracts key-value pairs (e.g., “Period Ending” → “7/18/2008”).
- **Tables Extraction** → detects rows, columns, and line items (e.g., earnings, deductions).
- **Queries** → allows custom questions like “What is the year-to-date gross pay?” → returns the value.
- **Expense Analysis** → extracts vendor names, line items, and prices from receipts/invoices.
- **ID Document Analysis** → extracts standardized fields (name, address, document number).

#### Use Cases

- **Financial Services** → process invoices, paystubs, financial reports.
- **Healthcare** → extract data from medical records and insurance claims.
- **Public Sector** → automate tax forms, passports, and ID verification.

#### Hands-On Demo Highlights

- Paystub example: detects raw text, layout, forms, and tables.
- Queries: “What is the regular hourly rate?” → Textract returns 10.
- Expense analysis: vendor + line item extraction.
- ID analysis: extracts first name, last name, city, address, document number.

#### Amazon Kendra Overview

- Fully managed **intelligent document search service** powered by ML.
- Indexes documents from multiple sources: text, PDF, Word, PowerPoint, HTML, FAQs, etc.
- Builds a **knowledge index** to enable natural language search.



## Core Capabilities

- **Natural Language Search** → users can ask questions like “Where is the IT support desk?” → Kendra answers “1st floor.”
- **Incremental Learning** → improves search results based on user interactions and feedback.
- **Fine-Tuned Search** → customize ranking based on importance, freshness, or filters.

## Supported Data Sources

- Text documents
- PDFs
- Microsoft Word
- PowerPoint
- HTML pages
- FAQs

## Use Cases

- **Enterprise search** → employees quickly find answers across internal documents.
- **Customer support** → users query FAQs and manuals directly.
- **Knowledge management** → unify scattered information into one searchable index.

## Amazon Mechanical Turk Overview

- Crowdsourcing marketplace to perform **simple human tasks**.
- Provides access to a **distributed virtual workforce** worldwide.
- Named after the 18th-century “Mechanical Turk” chess-playing automaton (which was secretly operated by a human).

## Core Capabilities

- **Image classification** → humans label datasets (e.g., millions of images).
- **Data collection** → gather or validate information.
- **Business processing** → repetitive, distributed tasks.
- **Recommendation review** → humans validate AI outputs.

## Pricing Model

- Requesters set a **reward per task** (e.g., \$0.10 per image).
- Large datasets can be processed quickly by many workers in parallel.
- Example: 10 million images × \$0.10 = \$1 million total cost.

## Integration with AWS AI Services

- Works with **Amazon SageMaker Ground Truth** for labeling datasets.
- Supports **Amazon Augmented AI (A2I)** for human review of ML predictions.

## Use Cases

- Training ML models with **labeled datasets**.
- Validating AI outputs (e.g., moderation, recommendations).
- Performing repetitive, distributed microtasks at scale.

## Amazon Augmented AI (A2I) Overview

- Provides **human oversight** for ML predictions in production.
- Ensures models are working correctly by routing low-confidence predictions to human reviewers.
- Integrates with AWS AI services (e.g., **Rekognition**) or custom ML models (e.g., **SageMaker**).

## Workflow

1. **Input data** → processed by ML model.
2. **High-confidence prediction** → returned directly to client application.
3. **Low-confidence prediction** → sent to **human review**.
4. **Human reviewers** consolidate predictions, generate risk-weighted scores.
5. **Reviewed outputs** → stored, returned to client, and fed back into ML model for continuous improvement.

## Human Review Options

- **Internal employees** → for sensitive/confidential data.
- **AWS Mechanical Turk workforce** → large-scale distributed workforce.
- **Pre-screened vendors** → meet confidentiality requirements.

## Use Cases

- **Content moderation** → ensure flagged images/videos are reviewed before final decision.
- **Document processing** → validate extracted data from Textract.
- **Fraud detection** → human oversight for suspicious transactions.
- **Medical AI** → confirm predictions in healthcare scenarios.

## Key Exam Point

- A2I is about **human-in-the-loop ML**.
- It's not a standalone ML service but a **review layer** that integrates with other AWS AI services or custom models.

## Amazon Augmented AI (A2I) Overview

- Part of **Amazon SageMaker**.
- Provides **human review workflows** for ML predictions.
- Ensures low-confidence predictions are validated by humans before being returned to applications.

## Supported Task Types

- **Textract Review** → validate key-value pair extraction from documents.
- **Rekognition Review** → confirm image moderation results (e.g., explicit or violent content).
- **Custom Workflow** → design your own review process for custom ML models.

## Confidence-Based Routing

- **High confidence predictions** → returned directly to client application.
- **Low confidence predictions** → sent to human reviewers.
- Option to **randomly sample** (e.g., 5% of predictions) for human review regardless of confidence.

## Human Review Workforce Options

- **Amazon Mechanical Turk** → over 500,000 independent contractors.
- **Private teams** → internal employees for sensitive data.
- **Third-party vendors** → pre-screened providers via AWS Marketplace.
- Pricing per task can be set (e.g., \$0.20–\$1.00 per review).

## Workflow Example: Image Moderation

1. Input image analyzed by **Amazon Rekognition**.
2. If confidence score < threshold (e.g., 50%), send to A2I.
3. Human reviewers check categories (nudity, violence, offensive content).
4. Reviewed results returned to client application.
5. Feedback loop → reviewed outputs improve ML model accuracy.

## Key Exam Point

- A2I = **human-in-the-loop ML**.
- Used when **human oversight is required** for sensitive or low-confidence predictions.

## AI Services for Medical Space

### Amazon Transcribe Medical

- Specialized version of Transcribe for **medical speech-to-text**.
- **HIPAA compliant** → safe for regulated healthcare environments.
- Recognizes **medical terminologies** (drug names, procedures, conditions).
- Modes:
  - **Real-time transcription** (e.g., physician dictation).
  - **Batch transcription** (e.g., phone call recordings).
- **Use Cases:**
  - Dictation of medical notes.

- Transcription of drug safety/side effect reports.

### Amazon Comprehend Medical

- Specialized version of Comprehend for **medical text analysis**.
- Extracts structured information from **clinical notes, discharge summaries, test results**.
- Detects **Protected Health Information (PHI)** for compliance.
- Can process data from **Amazon S3** or real-time streams via **Kinesis Data Firehose**.
- **Use Cases:**
  - Identify patient demographics (age, profession).
  - Extract medication details (name, dosage, frequency).
  - Structure unstructured medical text into usable data.

### Integration Flow

1. Audio → **Transcribe Medical** → text.
2. Text → **Comprehend Medical** → structured insights.
3. Output → used for **clinical documentation, compliance, or analytics**.

### Key Exam Point

- Both services are **medical-specific versions** of general AWS AI services.
- Designed for **healthcare compliance (HIPAA/PHI)** and **domain-specific accuracy**.

### Amazon Transcribe Medical (Speech-to-Text for Healthcare)

- Specialized version of **Amazon Transcribe** designed for medical environments.
- **HIPAA compliant** → safe for regulated healthcare use.
- Recognizes **medical terminology** (drug names, procedures, conditions).
- Modes:
  - Real-time transcription (e.g., physician dictation).
  - Batch transcription (e.g., call recordings).
- **Use Cases:** dictating medical notes, transcribing drug safety calls.

### Amazon Comprehend Medical (Text Analysis for Healthcare)

- Specialized version of **Amazon Comprehend** for medical text.
- Extracts **entities and relationships** from unstructured notes.
  - Patient demographics (age, gender, profession).
  - Medications (name, dosage, frequency, route).
  - Symptoms and conditions.
- Detects **Protected Health Information (PHI)** for compliance.
- Works with **Amazon S3** or real-time streams via **Kinesis Data Firehose**.

### Hands-On Demo Highlights

- Input: compressed doctor notes (e.g., “PT is 87 YO, PO QHS”).
- Output: structured insights → age = 87, gender = female, profession = teacher, medication details linked to dosage and route.
- Shows how **relationships** are identified (e.g., dosage tied to specific drug).
- Integration flow:
  1. Audio → **Transcribe Medical** → text.
  2. Text → **Comprehend Medical** → structured insights.

### Key Exam Point

- These are **medical-specific versions** of AWS AI services.
- Designed for **healthcare compliance (HIPAA/PHI)** and **domain-specific accuracy**.

### AWS HealthScribe Overview

- HIPAA-eligible service designed for **clinical documentation automation**.
- Converts **patient-clinician conversations** into structured clinical notes.
- Built into **Amazon Transcribe Medical** (currently available in **us-east-1 / Northern Virginia**).

### Core Capabilities

- **Rich transcripts** → captures full dialogue between patient and clinician.

- **Speaker role identification** → distinguishes patient vs clinician speech.
- **Dialogue classification** → organizes conversation into meaningful sections.
- **Medical term extraction** → identifies conditions, medications, symptoms.
- **Clinical note generation** → produces structured notes (chief complaint, assessment, plan).

### Hands-On Demo Highlights

- Upload audio (e.g., diabetes or blood pressure consultation).
- Customize vocabulary for medical terms.
- Output includes:
  - Transcript with speaker roles.
  - Insights section: chief complaint, assessment, plan.
  - Example: “Chief complaint = tiredness” → structured into clinical notes.

### Use Cases

- **Reduce documentation time** → automate physician note-taking.
- **AI-generated transcripts** → improve accuracy and consistency.
- **Efficient patient visit recap** → structured summaries for follow-up care.

### Key Exam Point

- HealthScribe is **not a standalone console yet** → accessed via **Amazon Transcribe Medical**.
- Designed specifically for **healthcare compliance and productivity**.

### Amazon EC2 Overview

- EC2 = **Elastic Compute Cloud**, AWS’s core **Infrastructure-as-a-Service (IaaS)** offering.
- Provides **virtual servers (instances)** with configurable CPU, memory, storage, and networking.
- Supports multiple OS (Linux, Windows, macOS).
- Key components:
  - **EBS drives** → attachable storage volumes.
  - **Elastic Load Balancer (ELB)** → distribute traffic across instances.
  - **Auto Scaling Groups (ASG)** → automatically scale capacity.
  - **Security Groups** → firewall rules for inbound/outbound traffic.
  - **User Data** → scripts/configuration at instance launch.

### Instance Types for AI/ML

- **GPU Instances** → families like **P3, P4, P5, G3–G6** optimized for ML training/inference.
- **Trainium Instances** → Trn1 family with AWS Trainium chips.
  - Designed for **deep learning training** (100B+ parameter models).
  - Up to **50% cost reduction** vs GPUs.
- **Inferentia Instances** → Inf1/Inf2 families with AWS Inferentia chips.
  - Optimized for **inference workloads**.
  - Up to **4× throughput** and **70% cost reduction** compared to GPUs.
- Both Trainium and Inferentia → **lowest environmental footprint** due to efficiency.

### AI Relevance

- While many AI workloads use **managed services** (e.g., SageMaker, Rekognition, Comprehend), EC2 is still critical when:
  - Running **custom ML frameworks**.
  - Training **large-scale models** directly on GPU/Trainium instances.
  - Serving models with **Inferentia** for cost-effective inference.

### Key Exam Point

- EC2 = virtual servers in the cloud.
- AI-related exam focus → **GPU families, Trainium (Trn1), Inferentia (Inf1/Inf2)**.
- Remember: Trainium = training, Inferentia = inference.

### Amazon EC2 Hands-On for Machine Learning

When launching an EC2 instance, the **instance type** selection is critical for ML workloads.

- **Instance Type Advice Tool**

- You can request recommendations based on workload (e.g., **deep learning inference**).
- Example suggestions: **G5g, C7gn**.
- These families vary in vCPUs, networking, and GPU availability.
- **GPU-Based Instances**
  - Documentation highlights GPUs like **NVIDIA T4G Tensor Core GPU** for ML acceleration.
  - Not always shown directly in the console advice table, but available in AWS docs.
- **Trainium Instances (Trn1)**
  - Purpose: **deep learning training**.
  - Example: **Trn1.32xlarge** → contains 16 Trainium accelerators.
  - Cost: ~\$21/hour → very powerful but expensive.
  - Best for **large-scale model training** (100B+ parameters).
- **Inferentia Instances (Inf1/Inf2)**
  - Purpose: **inference workloads**.
  - Example: **Inf2.48xlarge** → optimized for high-throughput inference.
  - Cost: ~\$14/hour.
  - Delivers **4× throughput** and **70% cost reduction** compared to GPU instances.
- **Key Hands-On Takeaway**
  - **Training** → use **Trn1 (Trainium)**.
  - **Inference** → use **Inf1/Inf2 (Inferentia)**.
  - **General ML workloads** → GPU families (P, G series).
  - Always check **pricing** before launching—these instances are powerful but costly.

## Amazon SageMaker Overview

- Fully managed service for **end-to-end machine learning workflows**.
- Designed for **data scientists and ML engineers** to build, train, and deploy ML models at scale.
- Central to the **AWS Certified Machine Learning Specialty exam**, but also relevant at the **AI Practitioner level**.

## Core Capabilities (Exam-Level)

- **Data Preparation** → import, clean, and label datasets.
- **Model Building** → use built-in algorithms or custom code.
- **Training** → run distributed training jobs on optimized EC2 instances (GPU, Trainium).
- **Deployment** → host models with auto-scaling endpoints.
- **MLOps Integration** → pipelines for CI/CD of ML models.
- **Experiment Tracking** → compare model versions and metrics.
- **Marketplace** → access pre-trained models and algorithms.

## Special Features

- **SageMaker Studio** → unified IDE for ML workflows.
- **SageMaker Autopilot** → automatically builds, trains, and tunes models.
- **SageMaker Ground Truth** → data labeling with human-in-the-loop.
- **SageMaker Clarify** → bias detection and explainability.
- **SageMaker Debugger** → real-time training insights.
- **SageMaker Model Monitor** → detect data drift in production.

## Amazon SageMaker Overview

- Fully managed service for **end-to-end machine learning workflows**.
- Provides a single platform to **collect data, build models, train, tune, deploy, and monitor**.
- Removes complexity of provisioning compute resources for ML.

## Core Capabilities

- **Data preparation** → import, clean, and transform datasets.
- **Built-in algorithms** → regression, classification, KNN, PCA, K-means, anomaly detection, NLP, image classification.

- **Automatic Model Tuning (AMT)** → optimizes hyperparameters automatically with early stopping to save cost.
- **Deployment options** → real-time, serverless, asynchronous, batch transform.
- **SageMaker Studio** → unified IDE for ML development, debugging, collaboration, and deployment.

#### Deployment Types (Exam Focus)

- **Real-time inference** → low latency, small payloads (<6 MB), immediate response.
- **Serverless inference** → no infrastructure management, low latency but possible cold start.
- **Asynchronous inference** → near-real time, large payloads (up to 1 GB), longer processing (≤1 hour). Request and response are in Amazon S3
- **Batch transform** → high latency, multiple records processed concurrently, suitable for datasets.

| Inference Type         | Latency                         | Payload Size                                 | Processing Time | Use Case                                                                        |
|------------------------|---------------------------------|----------------------------------------------|-----------------|---------------------------------------------------------------------------------|
| Real-time Inference    | Low (milliseconds to seconds)   | Up to 25 MB (one record)                     | Max 60 seconds  | Fast, near-instant predictions for web/mobile apps                              |
| Serverless Inference   | Low (milliseconds to seconds)   | Up to 4 MB (one record)                      | Max 60 seconds  | Sporadic, short-term inference without infrastructure, can tolerate cold starts |
| Asynchronous Inference | Medium to High "near real-time" | Up to 1 GB (one record)                      | Max 1 hour      | Large payloads and workloads requiring longer processing times                  |
| Batch Transform        | High (minutes to hours)         | Up to 100 MB per invocation (per mini batch) | Max 1 hour      | Bulk processing for large datasets<br>Concurrent processing                     |

#### Hands-On Practical Flow

1. **Launch SageMaker Studio** → central workspace for ML.
2. **Prepare dataset** → e.g., exam scores, IT/AWS experience, study hours.
3. **Train model** → choose algorithm (linear regression, classification, etc.).
4. **Use AMT** → optimize hyperparameters automatically.
5. **Deploy model** → select endpoint type (real-time, serverless, async, batch).
6. **Monitor predictions** → track performance, retrain if needed.

#### Key Exam Keywords

- “No infrastructure” → **Serverless inference**.
- “Near-real time” → **Asynchronous inference**.
- “Multiple records/dataset” → **Batch transform**.
- “Low latency, small payload” → **Real-time inference**.

#### Amazon SageMaker Studio Overview

- SageMaker has two entry points: **SageMaker** (higher-level) and **SageMaker AI** (full ML development).
- **SageMaker AI** is where you **build, train, and deploy ML models at scale**.
- Setup involves creating a **SageMaker domain** and a **Studio user profile**.

#### SageMaker Studio Capabilities

- **Applications** → RStudio, Canvas, MLFlow, Jupyter Lab.
- **Models** → access JumpStart base models (e.g., DeepSeek R1, Meta Llama), customize via UI or code.
- **Datasets** → upload and manage training data.
- **Experiments** → track ML experiments, training jobs, inference optimization, evaluation.
- **Pipelines** → visual editor for ML workflows and CI/CD automation.
- **Feature Store** → manage and reuse features across ML models.

## Hands-On Practical Flow in Studio

1. **Launch SageMaker Studio** → unified ML workspace.
2. **Explore JumpStart models** → quickly deploy base models like Llama or DeepSeek.
3. **Upload datasets** → prepare training data.
4. **Run experiments** → train, evaluate, and optimize models.
5. **Use pipelines** → automate workflows visually.
6. **Deploy models** → choose endpoint type (real-time, serverless, async, batch).
7. **Monitor performance** → track predictions, retrain if needed.

## Key Exam Point

- **SageMaker AI** = the full ML platform.
- **SageMaker Studio** = the interface where all ML tasks (data prep, training, tuning, deployment, monitoring) are accessible in one place.
- Know the difference between **JumpStart models** (pre-built, customizable) and **custom models**

## Amazon SageMaker Data Wrangler (Now fully integrated in SageMaker Canvas)

- Tool for **data preparation**, transformation, and feature engineering.
- Supports **tabular and image data**.
- Features:
  - **Data selection** → import from sources like Amazon S3.
  - **Data cleansing** → handle missing values, correct formats.
  - **Data exploration** → preview datasets, inspect column types.
  - **Visualization** → graphs to understand distributions and patterns.
  - **Transformations** → apply functions, drop/add columns.
  - **SQL support** for advanced users.
  - **Data quality analysis** → detect anomalies, missing values, incorrect formats.
  - Export workflows → integrate into **SageMaker Pipelines** for automation.

## Feature Engineering

- Process of creating **features** (inputs for ML models).
- Example: Convert **birth date** → **age** (numerical value).
- Other examples: song rating, listening duration, demographics.
- High-quality features improve both **training** and **inference**.

## SageMaker Feature Store

- Central repository for **features** across datasets.
- Ingest features from multiple sources.
- Features are **discoverable and reusable** across teams.
- Integration: publish features directly from **Data Wrangler**.
- Enables **collaboration and consistency** in ML workflows.

## Hands-On Practical Flow

1. Import dataset into **Data Wrangler**.
2. Cleanse and transform data (e.g., handle missing values, normalize).
3. Perform **feature engineering** (e.g., derive age from birth date).
4. Analyze data quality and visualize distributions.
5. Export workflow → integrate into **SageMaker Pipelines**.
6. Publish features into **Feature Store** for reuse across ML models.

## Key Exam Keywords

- “Data preparation & transformation” → **Data Wrangler**.
- “Reusable features across datasets” → **Feature Store**.
- “Convert raw data into usable inputs” → **Feature engineering**.

## SageMaker Clarify Overview

- Provides tools for **model evaluation**, **bias detection**, and **explainability**.

- Helps compare **foundation models** (e.g., Model A vs Model B) on human-centric tasks like friendliness, humor, or relevance.
- Uses **human reviewers** (AWS-managed teams, employees, or Mechanical Turk workers) to evaluate models.
- Supports **built-in datasets and metrics**, or custom datasets/questions.

### Key Features

- **Model Comparison** → Evaluate multiple models on tasks (e.g., Model A = 25% brand voice vs Model B = 75%).
- **Bias Detection** → Identify dataset/model bias (e.g., class imbalance, gender distribution issues).
- **Explainability** → Understand why a model predicts certain outcomes. Example: loan rejection explained by features like maturity month or loan amount.
- **Trust & Debugging** → Increase confidence in ML predictions by showing feature importance and reasoning.

### SageMaker Ground Truth Overview

- Provides **human-in-the-loop data labeling** and **reinforcement learning from human feedback (RLHF)**.
- Aligns models to **human preferences** by incorporating feedback into the reward function.
- Use cases:
  - **Model review & customization** → fine-tune models to match business tone (e.g., serious vs joyful).
  - **Data annotation** → humans label datasets (images, text, etc.).
- Workforce options: employees, third-party vendors, or **Amazon Mechanical Turk**.
- **Ground Truth Plus** → managed workforce option for large-scale labeling tasks.

### Hands-On Practical Flow

1. Use **Clarify** to compare foundation models on tasks (friendliness, humor, relevance).
2. Apply **bias detection** to datasets and models.
3. Use **explainability tools** to understand predictions and feature importance.
4. Apply **Ground Truth** for RLHF → collect human feedback to align models.
5. Use **Ground Truth Plus** for scalable, managed data labeling.

### Key Exam Keywords

- “Model comparison, bias detection, explainability” → **Clarify**.
- “RLHF, human feedback, data labeling” → **Ground Truth**.
- “Managed workforce for labeling” → **Ground Truth Plus**.

### SageMaker Governance Tools

- **Model Cards** → Document essential model information in one place.
  - Intended uses, risk rating, training details.
  - Helps ensure transparency and accountability.
- **Model Dashboard** → Centralized repository of all deployed models.
  - Shows risk rating, model quality, data quality.
  - Tracks which models are actively serving inference.
  - Integrates with **Model Monitor** to detect violations (bias, drift, quality issues).
- **Model Monitor** → Continuous or scheduled monitoring of deployed models.
  - Detects **data drift** or **model drift**.
  - Example: Loan approval model starts approving applicants with poor credit scores → flagged for retraining.
- **Role Manager** → Define permissions for personas (data scientists, MLOps engineers, etc.).
  - Ensures proper governance and access control.
- **Model Registry** → Catalog for tracking, managing, and versioning ML models.
  - Stores metadata, versions, and approval status.
  - Supports governance workflows (e.g., steward approval before deployment).



- Useful for **automated deployments** and sharing models across teams.
- **SageMaker Pipelines** → Automates ML workflows (CI/CD for ML).
  - Steps include:
    - **Processing** → data prep/feature engineering.
    - **Training** → model training.
    - **Tuning** → hyperparameter optimization.
    - **AutoML** → automated training.
    - **Model** → register model in registry.
    - **ClarifyCheck** → bias/drift/explainability checks.
    - **QualityCheck** → data/model quality validation.
  - Benefits: faster iteration, reduced errors, repeatable automation.

### Hands-On Practical Flow for Governance

1. Document model details in **Model Cards**.
2. Track all models in **Model Dashboard**.
3. Set up **Model Monitor** for drift detection.
4. Use **Role Manager** to enforce access policies.
5. Register models in **Model Registry** with approval workflows.
6. Automate end-to-end ML lifecycle with **Pipelines**.

### Key Exam Keywords

- “Transparency, intended use, risk rating” → **Model Cards**.
- “Centralized repository, monitoring quality” → **Model Dashboard**.
- “Detect drift, continuous monitoring” → **Model Monitor**.
- “Permissions for personas” → **Role Manager**.
- “Versioning, approval status” → **Model Registry**.
- “Automated ML workflows, CI/CD” → **Pipelines**.

### SageMaker JumpStart

- Acts as a **Machine Learning Hub**.
- Provides access to **pre-trained foundation models** (CV, NLP) from Hugging Face, Databricks, Meta, Stability AI, etc.
- Models can be **customized with your own data** and deployed directly on SageMaker.
- Also offers **pre-built ML solutions** for common business use cases (fraud detection, demand forecasting, credit scoring, computer vision).

### SageMaker Canvas

- A **no-code interface** for building ML models.
- Allows users to create ML pipelines visually without writing code.
- Powered by **SageMaker Autopilot** (AutoML behind the scenes).
- Integrates with **Data Wrangler** for transformations.
- Direct integration with AWS AI services:
  - **Comprehend** → sentiment analysis.
  - **Rekognition** → object detection.
  - **Textract** → document extraction.

### MLFlow on SageMaker

- Open-source tool for **ML lifecycle management**.
- Integrated into SageMaker Studio.
- Supports **tracking experiments, runs, and metrics**.
- Can launch an **MLFlow Tracking Server** directly from SageMaker.
- Provides flexibility for teams already using MLFlow in their workflows.

### Hands-On Practical Flow

1. Use **JumpStart** to quickly launch pre-trained models or pre-built solutions.
2. If you prefer no-code, use **Canvas** to visually build ML pipelines.
3. For open-source workflows, integrate **MLFlow** to track experiments and manage lifecycle.
4. Deploy models directly in SageMaker with full control over endpoints.

## Key Exam Keywords

- “Pre-trained models, ML hub” → **JumpStart**.
- “No-code interface, AutoML” → **Canvas**.
- “Open-source lifecycle management” → **MLFlow**.

## Extra Features in SageMaker (Exam-Relevant)

- **Network Isolation Mode**
  - Ensures **maximum security** by preventing SageMaker training job containers from accessing the internet.
  - Protects against data leakage during training.
  - Jobs cannot access **Amazon S3, VPC resources, or external networks** when isolation is enabled.
  - Exam keyword: “*no outbound internet access*”.
- **DeepAR Forecasting Algorithm**
  - Specialized algorithm for **time series forecasting**.
  - Uses **Recurrent Neural Networks (RNNs)** to predict future values.
  - Example use cases: demand forecasting, financial predictions, resource planning.
  - Exam keyword: “*DeepAR → time series forecasting with RNN*”.

## Hands-On Practical Flow

1. When launching a training job, enable **Network Isolation** for sensitive datasets.
2. For forecasting tasks (sales, demand, stock prices), select **DeepAR** as the algorithm.
3. Configure hyperparameters (e.g., context length, prediction length) and train the model.
4. Deploy the model for real-time or batch predictions depending on use case.

## Key Exam Keywords

- “Network isolation → no internet access for training jobs.”
- “DeepAR → time series forecasting using RNN.”

## Responsible AI Overview

- **Fairness** → promote inclusion, prevent discrimination.
- **Explainability** → understand model behavior and outputs.
- **Privacy & Security** → individuals control when/if their data is used.
- **Transparency** → clear documentation of models and processes.
- **Veracity & Robustness** → reliable systems even in unexpected situations.
- **Governance** → policies, oversight, alignment with regulations.
- **Safety** → ensure algorithms are beneficial to society.
- **Controllability** → align models with human values and intents.

## AWS Tools for Responsible AI

- **Amazon Bedrock Guardrails** → filter harmful content, redact PII, block undesirable topics.
- **SageMaker Clarify** → evaluate foundation models, detect bias, explain predictions.
- **SageMaker Data Wrangler** → fix bias via data augmentation, balance datasets.
- **SageMaker Model Monitor** → monitor model quality in production.
- **Amazon Augmented AI (A2I)** → human review for low-confidence predictions.
- **SageMaker Role Manager** → manage permissions and personas.
- **Model Cards & Model Dashboard** → document models, track risk ratings, monitor deployments.
- **AWS AI Service Cards** → official documentation of intended use cases, limitations, and design choices.

## Interpretability vs Explainability

- **Interpretability** → humans can understand *how* the model makes decisions.
  - Example: **Linear regression** → easy to interpret, low performance.
  - Example: **Neural networks** → high performance, poor interpretability.

- **Explainability** → ability to explain inputs/outputs without fully understanding internal mechanics.
  - Example: **Decision trees** → clear, visual, interpretable.
  - **Partial Dependence Plots (PDPs)** → show how one variable impacts predictions while holding others constant.

### Human-Centered Design (HCD) for Explainable AI

- **Amplified decision making** → clarity, simplicity, usability in high-pressure environments.
- **Unbiased decision making** → recognize and mitigate dataset/model bias.
- **Human & AI learning** → cognitive apprenticeship, RLHF, personalization.
- **User-centered design** → ensure accessibility and benefit for diverse users.
- 

### Capabilities vs Challenges

- **Capabilities:** adaptable, responsive, creative, efficient with data, scalable, personalized
- **Challenges:** regulation difficulties, social risks (misinformation), privacy concerns, toxicity, hallucinations, plagiarism, prompt misuse, exposure, jailbreaking

### Toxicity

- Offensive or disturbing outputs
- Hard to define boundaries between toxic vs informative content
- Mitigation: curate training data, apply guardrails to filter unwanted content

### Hallucinations

- Model generates plausible but incorrect information
- Example: claiming authorship of books that do not exist
- Cause: probabilistic next-word sampling
- Mitigation: educate users, verify with independent sources, mark outputs as unverified

### Plagiarism and Cheating

- AI can generate essays, reports, or job application samples
- Difficult to trace sources of generated content
- Detection technologies are emerging to identify AI-generated text/images

### Prompt Misuse

- **Poisoning:** biased or malicious data introduced into training sets
- **Hijacking/Injection:** prompts crafted to manipulate outputs (e.g., misinformation, malicious code)
- **Exposure:** sensitive data revealed during inference or training
- **Prompt leaking:** disclosure of confidential prompts or inputs

### Jailbreaking

- Circumventing ethical and safety constraints of models
- Techniques: zero-shot, few-shot, and many-shot prompting
- Many-shot jailbreaking can bypass restrictions and force harmful outputs

### Compliance Challenges in AI

- **Complexity & opacity** → hard to audit AI decisions.
- **Dynamism & adaptability** → models evolve over time.
- **Emergent capabilities** → unintended behaviors.
- **Risks** → bias, privacy violations, misinformation.
- **Algorithm accountability** → transparency and explainability required.
- **Regulations** → EU AI Act, US state-level laws.

### AWS Compliance Frameworks

- Over **140 certifications** including **NIST, ENISA, ISO, SOC, HIPAA, GDPR, PCI DSS**.
- **Model Cards** → document datasets, sources, biases, training details, risk ratings.
- **Service Cards** → AWS-provided responsible AI documentation for services like Textract, Rekognition.

## Key Exam Keywords

- “Bias detection, explainability” → **Clarify**.
- “Data augmentation for fairness” → **Data Wrangler**.
- “Human review for low confidence” → **A2I**.
- “Governance, permissions” → **Role Manager**.
- “Documentation, transparency” → **Model Cards / Service Cards**.
- “Monitoring drift” → **Model Monitor**.
- “Compliance frameworks” → **HIPAA, GDPR, PCI DSS, ISO, SOC**.

## Why Governance & Compliance Matter in AI

- **Trust building** → Governance ensures AI systems are transparent and reliable, fostering user confidence.
- **Risk mitigation** → Helps prevent bias, privacy violations, and unintended consequences.
- **Legal alignment** → Ensures adherence to regulatory frameworks, reducing exposure to legal or reputational risks.
- **Public confidence** → Responsible deployment builds societal trust in AI solutions.

## Governance Framework Example

- **AI Governance Board** → Includes legal, compliance, privacy, and AI subject matter experts.
- **Defined roles** → Oversight, policy-making, risk assessment, decision-making.
- **Policies & procedures** → Cover full AI lifecycle: data management, model training, deployment, monitoring.
- **AWS Tools for Governance:**
  - **AWS Config** → track configurations.
  - **Amazon Inspector** → security assessments.
  - **AWS Audit Manager** → compliance audits.
  - **AWS Artifacts** → access compliance reports.
  - **AWS CloudTrail** → log API activity.
  - **AWS Trusted Advisor** → best practice checks.

## Governance Strategies

- **Policies** → principles for data management, training, validation, safety, oversight, IP, bias mitigation, privacy.
- **Review cadence** → monthly/quarterly/annual reviews with SMEs, legal, compliance, and end users.
- **Review strategy** → technical (performance, quality, robustness) + non-technical (policies, regulations).
- **Transparency standards** → publish training data, decisions, limitations, and use cases.
- **Feedback channels** → allow stakeholders to raise concerns.
- **Team training** → bias mitigation, responsible AI practices, certifications, cross-functional collaboration.

## Data Governance Strategies

- **Data governance council** → define roles for stewards, owners, custodians.
- **Data sharing agreements** → secure collaboration without compromising ownership.
- **Data lifecycle management** → collection, processing, storage, consumption, archival.
- **Data logging** → track inputs, outputs, metrics, events.
- **Data residency** → ensure compliance with location-based regulations.
- **Data monitoring** → detect anomalies and drift.
- **Data retention** → balance regulatory requirements with cost.
- **Data lineage** → document sources, licenses, transformations, catalog datasets for transparency and accountability.

## Key Exam Keywords

- “Governance board, oversight, policies” → **Governance framework**.
- “Bias, privacy, unintended consequences” → **Risk mitigation**.

- “Transparency, documentation, feedback channels” → **Governance strategies.**
- “Data lifecycle, residency, lineage” → **Data governance strategies.**
- “AWS Config, Inspector, Audit Manager, CloudTrail” → **AWS governance tools.**

## Security & Privacy in AI Systems

### Threat Detection

- Detect fake content, manipulated data, or automated attacks.
- Use AI-based detection systems to analyze **network traffic**, **user behavior**, and other signals.

### Vulnerability Management

- Regular **security assessments**, penetration testing, and code reviews.
- Strong **patch management** and update processes to fix software bugs or model weaknesses.

### Infrastructure Protection

- Secure **cloud platforms**, **edge devices**, and **data stores**.
- Implement **access control**, **network segmentation**, and **data encryption**.
- Ensure resilience against **system failures**.

### Prompt Injection Defense

- Guardrails: prompt filtering, sanitization, and validation.
- Prevent malicious instructions from bypassing ethical boundaries.

### Data Encryption

- Encrypt data **at rest** and **in transit**.
- Manage encryption keys securely to prevent unauthorized access.

### Monitoring AI Systems

- Track **model accuracy**, **precision**, **recall**, **F1 score**, **latency**.
- Monitor infrastructure: CPU/GPU usage, network performance, storage, logs.
- Watch for **bias**, **fairness**, **compliance issues**.

### Shared Responsibility Model

- **AWS responsibility** → security *of* the cloud (hardware, software, facilities, networking).
- **User responsibility** → security *in* the cloud (data management, access controls, guardrails, encryption).
- **Shared controls** → patch management, configuration, training.

### Secure Data Engineering Practices

- **Data quality checks** → completeness, accuracy, timeliness, consistency.
- **Data profiling & lineage** → track sources, transformations, and usage.
- **Privacy-enhancing technologies** → masking, obfuscation, encryption, tokenization.
- **Access control** → role-based access, fine-grained permissions, SSO, MFA, IAM.
- **Audit & monitoring** → log access activities, enforce least privilege.
- **Data integrity** → backups, recovery strategies, audit trails, consistency checks.

### Key Exam Keywords

- “Fake content detection” → **Threat detection.**
- “Patch management, penetration testing” → **Vulnerability management.**
- “Access control, encryption, segmentation” → **Infrastructure protection.**
- “Guardrails, sanitization” → **Prompt injection defense.**
- “Shared responsibility model” → AWS vs user security roles.
- “Data masking, tokenization, lineage” → **Secure data engineering.**

## Generative AI Security Scoping Matrix

- Framework to classify GenAI applications by **ownership level** and associated **security risks**.
- **Consumer apps** → use public GenAI services (ChatGPT, Midjourney). Lowest ownership, minimal control.
- **Enterprise apps** → SaaS GenAI features (Salesforce Einstein GPT, Amazon Q Developer). Moderate ownership with customization.
- **Pre-trained models** → use base models (Amazon Bedrock). Medium ownership, no training.

- **Fine-tuned models** → customize models with your own data (Bedrock customized, SageMaker JumpStart). Higher ownership, more responsibility.
- **Self-trained models** → train from scratch (SageMaker). Full ownership of algorithm, data, and infrastructure.
- **Security implications** → governance, compliance, privacy, risk management, and resilience vary depending on scope.

### Machine Learning Operations (MLOps)

- Extension of **DevOps** for ML systems. Ensures models are **deployed, monitored, retrained, and versioned** continuously.
- **Core principles:**
  - **Version control** → datasets, code, and models tracked for rollback.
  - **Automation** → pipelines for ingestion, preprocessing, training, deployment.
  - **Continuous integration** → test models consistently.
  - **Continuous delivery** → deploy models reliably.
  - **Continuous retraining** → adapt to new data and feedback.
  - **Continuous monitoring** → detect drift, bias, or performance issues.
- **Pipeline stages:**
  - Data preparation → Model build → Evaluation → Selection → Deployment → Monitoring.
- **Repositories:**
  - **Data repository** → versioned datasets.
  - **Code repository** → versioned ML code.
  - **Model registry** → versioned deployed models.

### Key Exam Keywords

- “Consumer vs enterprise vs fine-tuned vs self-trained” → **Security Scoping Matrix**.
- “Version control, automation, CI/CD, retraining, monitoring” → **MLOps**.
- “Model registry, pipelines, governance” → **MLOps infrastructure**.

### IAM (Identity and Access Management)

- **Global service** → manages users, groups, and permissions across AWS.
- **Root account** → created by default, only for initial setup, never shared or used for daily tasks.
- **Users** → represent individuals; each user gets specific permissions.
- **Groups** → collections of users (e.g., Developers, Operations). Groups cannot contain other groups.
- **Policies** → JSON documents defining what actions/services users or groups can access.
- **Least privilege principle** → grant only the minimum permissions needed.

### IAM User Creation (Practice)

- **Global service** → IAM users exist across all regions (no region selection).
- **Root account** → avoid daily use; create IAM users instead.
- **User creation** → assign username, console access, and password settings.
- **Groups** → create groups (e.g., Admins) and attach policies like **AdministratorAccess**. Users inherit group permissions.
- **Tags** → optional metadata (e.g., department = engineering).
- **Account alias** → customize sign-in URL for easier login.
- **Best practice** → use IAM user for daily tasks, not root.

### Multi-Session Support in AWS Console

- **Feature** → lets you sign into multiple AWS accounts/roles in the same browser.
- **Add session** → log in with another account ID or root credentials.
- **Parallel accounts** → manage different accounts side by side (e.g., one shows EBS volumes, the other doesn't).
- **Benefit** → simplifies working at scale, no need for separate browsers or private windows.

## IAM Policies Overview

- **Group policies** → attached at group level; all members inherit permissions.
- **Inline policies** → attached directly to a single user; independent of groups.
- **Multiple group membership** → users can inherit policies from multiple groups.

## Policy Structure (JSON)

- **Version** → policy language version (e.g., 2012-10-17).
- **ID** → optional identifier.
- **Statements** → one or more rules, each with:
  - **Sid** → optional statement ID.
  - **Effect** → Allow or Deny.
  - **Principal** → account, user, or role the policy applies to.
  - **Action** → list of API calls permitted/denied.
  - **Resource** → AWS resources affected (e.g., S3 bucket).
  - **Condition** → optional, defines when statement applies.

## IAM Policies (Hands-On)

- **Group inheritance** → users gain permissions from groups (e.g., Admin group → AdministratorAccess).
- **Removing user from group** → user loses inherited permissions (e.g., Stephane removed → access denied).
- **Direct policies** → attach policies directly to a user (e.g., IAMReadOnlyAccess → allows viewing but not creating groups).
- **Multiple sources** → a user can inherit permissions from groups + direct policies simultaneously.
- **Policy examples:**
  - **AdministratorAccess** → Action: \* + Resource: \* (full access).
  - **IAMReadOnlyAccess** → allows List\* and Get\* API calls only.
- **Policy creation** → via JSON editor or visual editor; select actions (e.g., ListUsers, GetUser) and resources.

## IAM Roles Overview

- **Purpose** → grant permissions to AWS services (not people) so they can act on your behalf.
- **Service usage** → EC2 instances, Lambda functions, CloudFormation stacks often need roles to access other AWS resources.
- **How it works** → role + service together form one entity; service uses the role's permissions to make API calls.
- **Difference from users** → roles are assumed by services or applications, not tied to a single person.

## IAM Roles (Hands-On)

- **Role creation** → done from IAM console under *Roles*.
- **AWS service role** → most common type; created for services like EC2 or Lambda.
- **Trusted entity** → defines which AWS service can assume the role (e.g., EC2).
- **Attach policy** → permissions assigned (e.g., IAMReadOnlyAccess).
- **Role name** → e.g., DemoRoleForEC2; appears in role list after creation.
- **Usage** → role is assumed by the service (EC2 instance) to perform actions with assigned permissions.

## Amazon S3 Overview

- **Core service** → infinitely scalable object storage, backbone for many websites and AWS services.
- **Use cases** → backup, disaster recovery, archival (Glacier), hybrid storage, hosting apps/media, data lakes, analytics, software delivery, static websites.
- **Examples** → NASDAQ uses **S3 Glacier** for 7 years of archival; Cisco runs analytics on S3 data.

## Buckets

- **Region-specific** → buckets tied to a region, though S3 has a global interface.
- **Naming rules** → lowercase only, no underscores/IPs, must not start with xn- or end with -s3alias.
- **Namespace update** → now supports account-regional namespace, so names don't need to be globally unique.

## Objects

- **Keys** → full path of file (prefix + object name). S3 doesn't truly have directories; console UI simulates them.
- **Size limits** → max 50 TB; files >5 GB require **multi-part upload**.
- **Metadata** → system/user-defined key-value pairs.
- **Tags** → up to 10 Unicode key-value pairs, useful for security/lifecycle.
- **Versioning** → optional, assigns version IDs to objects.

## Amazon S3 Bucket Creation (Hands-On)

- **Region-specific** → bucket created in a chosen region (e.g., eu-west-1 Ireland).
- **Bucket type** → General purpose (default, recommended) vs Directory (low-latency).
- **Namespace** → Global (requires unique name worldwide) vs Account Regional (recommended, suffix ensures uniqueness).
- **Object ownership** → ACL disabled by default for security.
- **Public access** → blocked by default for maximum security.
- **Encryption** → server-side encryption with S3 managed keys enabled.
- **Versioning** → disabled by default, can be enabled later.

## Uploading Objects

- **Upload file** → coffee.jpg uploaded successfully.
- **Access methods** →
  - Public URL → AccessDenied (blocked).
  - Pre-signed URL → works with embedded credentials.
- **Folders** → simulate directories (e.g., images folder with beach.jpg).
- **Delete objects** → deleting a folder removes all contained objects.

## Amazon S3 Storage Classes

### S3 Standard

- Availability: 99.99%
- Use: frequently accessed data, low latency, high throughput.
- Examples: big data analytics, mobile apps, content distribution.

### S3 Standard-Infrequent Access

- Availability: 99.9%
- Use: infrequently accessed but rapid retrieval needed.
- Examples: backups, disaster recovery.

### S3 One Zone-IA

- Availability: 99.5% (single AZ only).
- Use: secondary backups, recreatable data.

## Glacier Classes

- **Glacier Instant Retrieval** → millisecond retrieval, min storage 90 days.
- **Glacier Flexible Retrieval** → expedited (1–5 min), standard (3–5 hrs), bulk (5–12 hrs). Min storage 90 days.
- **Glacier Deep Archive** → lowest cost, retrieval standard (12 hrs) or bulk (48 hrs). Min storage 180 days.

## S3 Intelligent-Tiering

- Moves objects automatically between tiers based on usage.



- Frequent, infrequent, archive instant, archive access, deep archive access.
- Small monitoring fee, no retrieval charges.

### Durability & Availability

- **Durability: 11 nines (99.999999999%)** across all classes.
- That means if you store 10,000,000 objects with Amazon S3, we can on average expect to incur a loss of a single object once every 10,000 years.
- **Availability** varies: Standard (99.99%), IA (99.9%), One Zone-IA (99.5%).

### S3 Storage Classes (Hands-On Demo)

- **Bucket creation** → new bucket created (e.g., s3-storage-classes-demos-2022).
- **Upload object** → coffee.jpg uploaded with chosen storage class.
- **Storage class options** → Standard, Intelligent-Tiering, Standard-IA, One-Zone-IA, Glacier (Instant, Flexible, Deep Archive). Reduced Redundancy is deprecated.
- **Change class** → object storage class can be edited (e.g., Standard-IA → One-Zone-IA → Glacier → Intelligent-Tiering).
- **Lifecycle rules** → automate transitions between classes (e.g., Standard-IA after 30 days, Intelligent-Tiering after 60 days, Glacier after 180 days).

### Amazon EC2 Overview

- **Elastic Compute Cloud** → AWS's Infrastructure-as-a-Service (IaaS) offering.
- **EC2 instances** → virtual machines you rent; customizable OS (Linux, Windows, macOS), CPU, RAM, storage, and networking.
- **Storage options** →
  - **EBS volumes** (network-attached).
  - **Instance store** (hardware-attached).
  - **EFS** (shared file system).
- **Networking** → public IPs, fast network cards, firewall rules via **Security Groups**.
- **Scaling & load balancing** → Elastic Load Balancer (ELB) and Auto Scaling Groups (ASG).

### Bootstrapping with User Data

- **EC2 User Data** → script runs once at first launch.
- **Purpose** → automate boot tasks: install updates, software, download files, configure environment.
- **Runs as root** → commands have full privileges.

### Launching First EC2 Instance (Hands-On)

- **Instance setup** → name tag *My First Instance*, Amazon Linux 2 AMI (free tier eligible), architecture 64-bit x86.
- **Instance type** → T2 micro (free tier eligible, 750 hrs/month).
- **Key pair** → create RSA key pair (.pem for Linux/Mac/Win10, .ppk for older Windows).
- **Security group** → allow SSH (port 22) + HTTP (port 80) traffic.
- **Storage** → 8 GB gp2 root volume, delete on termination enabled.
- **User Data** → bootstrap script installs Apache (httpd), updates system, creates simple "Hello World" webpage.

### Instance Lifecycle

- **Start/Stop** → stop saves volume, no billing while stopped.
- **Terminate** → deletes instance + root volume (if delete on termination enabled).
- **Public vs Private IP** → public IP may change after stop/start; private IP remains constant.

## Web Server Demo

- Access via **public IPv4** with http:// (not HTTPS).
- Displays “Hello World” page showing **private IPv4** inside HTML.

## AWS Lambda Overview

- **Serverless compute** → no servers to manage, functions run on demand.
- **Event-driven** → triggered by AWS events (e.g., S3 upload, DynamoDB stream, CloudWatch schedule).
- **Scaling** → automatic, no need for Auto Scaling groups.
- **Pricing** → pay per request + compute time. Free tier: 1M requests/month + 400,000 GB-seconds.

## Benefits

- **Cost efficiency** → only billed when functions run.
- **Integration** → works with many AWS services (S3, DynamoDB, CloudWatch, EventBridge).
- **Languages** → Node.js, Python, Java, C#, Ruby, plus custom runtimes (Rust, Go).
- **Resource limits** → up to 10 GB RAM; increasing RAM also boosts CPU and network.

## Common Use Cases

- **Thumbnail creation** → S3 upload triggers Lambda → generates thumbnail → stores in S3 + metadata in DynamoDB.
- **Serverless CRON jobs** → CloudWatch Events/EventBridge trigger Lambda on schedule (e.g., hourly tasks).

## AWS Lambda Practice (Hands-On)

- **Lambda console intro** → UI shows Lambda functions can be written in multiple languages (Node.js, Python, Java, C#, Ruby, custom runtimes).
- **Event triggers** → Lambda responds to events (e.g., streaming analytics, mobile/IOT data, S3 uploads). Demonstrates automatic scaling with multiple invocations.
- **Create function** → use blueprint (Hello World), choose language (Python), name function (HelloWorld).
- **Execution role** → IAM role with basic Lambda permissions (e.g., CloudWatch Logs access).

## Function Code & Testing

- **Handler** → entry point for events; processes input JSON and returns output.
- **Test events** → pass JSON input, verify output; errors occur if expected keys are missing.
- **Monitoring** → CloudWatch shows invocations, logs, errors for debugging.

## Configuration & Permissions

- **General config** → set memory, ephemeral storage, timeout.
- **IAM role permissions** → role defines allowed actions/resources (e.g., CloudWatch Logs).
- **Triggers** → add event sources (S3 bucket events, DynamoDB streams, EventBridge schedules, etc.).

## Amazon Macie Overview

- **Purpose** → fully managed data security and privacy service.
- **How it works** → uses machine learning + pattern matching to discover sensitive data (e.g., PII) in **S3 buckets**.
- **Alerts** → notifies via **EventBridge**, can integrate with **SNS topics**, **Lambda functions**, etc.
- **Setup** → one-click enable, select S3 buckets to analyze.

## AWS Config Overview

- **Purpose** → records configuration changes of AWS resources, helps with **auditing** and **compliance tracking**.
- **Data storage** → configuration history stored in **S3**, can be analyzed with **Athena**.
- **Compliance checks** → Config rules detect issues like unrestricted SSH, public S3 buckets, or ALB changes.
- **Notifications** → alerts via **SNS**, integrates with **CloudTrail** to show who made changes.
- **Scope** → per-region service, but results can be aggregated across accounts and regions.

## Hands-On Example

- Enabled Config to record **all resources** in region.
- Applied **restricted-ssh rule** → detected 3/5 security groups noncompliant.
- Viewed **resource timeline** → showed configuration + compliance changes.
- Fixed by removing SSH open-to-all rule → resource became compliant.
- **Re-evaluation** confirmed compliance status updated.
- **CloudTrail integration** → showed root user removed the ingress rule.

## Amazon Inspector Overview

- **Purpose** → automated security assessments for EC2, ECR container images, and Lambda functions.
- **EC2 assessments** → uses Systems Manager agent to check unintended network accessibility + OS vulnerabilities.
- **ECR assessments** → scans container images (e.g., Docker) for known CVEs when pushed to Amazon ECR.
- **Lambda assessments** → scans function code + dependencies for vulnerabilities at deployment.

## Findings & Reporting

- **Security Hub integration** → centralizes vulnerability findings.
- **EventBridge integration** → enables automation workflows based on findings.
- **Continuous scanning** → re-runs when CVE database updates.
- **Risk scoring**

## AWS CloudTrail Overview

- **Purpose** → provides governance, compliance, and audit for AWS accounts.
- **Enabled by default** → automatically records API calls and events across the account.
- **Sources logged** → Console actions, SDK calls, CLI commands, and service activities.
- **Storage options** → logs can be sent to **CloudWatch Logs** or **Amazon S3** for retention and analysis.
- **Trails** → can be applied to all regions or scoped to a single region.

## Use Cases

- **Audit actions** → track who deleted resources, when, and from where.
- **Security monitoring** → detect unauthorized access or unusual API activity.
- **Compliance** → maintain records of IAM user/role activity for governance.

## AWS CloudTrail Hands-On

- **Event history** → shows last 90 days of management events (API calls, user activity).
- **EC2 demo** → terminated an EC2 instance; CloudTrail recorded the TerminateInstances API call.
- **Event details** → includes event source (EC2), access key used, region, and full event JSON.
- **Audit capability** → lets you verify who did what, when, and from where.

## AWS Artifact Overview

- **Portal purpose** → not a service, but a portal for compliance documentation and agreements.
- **Artifact Reports** → download AWS security/compliance documents (ISO, PCI, SOC reports) from third-party auditors.
- **Artifact Agreements** → review, accept, and track agreements (e.g., HIPAA, BAA) for accounts or organizations.
- **Third-party reports** → includes compliance reports from independent software vendors (ISVs) via AWS Marketplace Vendor Insights.
- **Notifications** → administrators can receive alerts when new reports are submitted.

## AWS Artifact Console (Hands-On)

- **Agreements** → shows compliance agreements for your account or organization. Example: *Australian Notifiable Data Breach Addendum* (inactive until accepted). You can **accept** or **download** agreements directly to support compliance.
- **Reports** → access 200+ AWS compliance reports (ISO, PCI, SOC, etc.). Reports can be downloaded for audits. If subscribed to ISVs via Marketplace Vendor Insights, their reports appear here too.
- **Notifications** → administrators can configure notifications to be alerted when new AWS or third-party compliance reports are published.

## AWS Audit Manager Overview

- **Purpose** → helps assess risk and compliance of AWS workloads, continuously audits service usage, and prepares for external audits.
- **Frameworks** → supports compliance with GDPR, HIPAA, PCI, SOC 2, and other standards.
- **Evidence collection** → automatically gathers compliance evidence across AWS resources into evidence folders.
- **Continuous auditing** → ensures ongoing compliance as workloads evolve.

## Workflow

- **Select frameworks** → choose compliance frameworks relevant to your organization.
- **Define scope** → specify accounts, regions, and services to audit.
- **Automated evidence** → collects logs, configurations, and compliance data continuously.
- **Control reviews** → validate compliance controls, delegate reviews to resource owners.
- **Reports** → generate audit-ready reports with findings and remediation actions.

## AWS Trusted Advisor Overview

- **Purpose** → provides high-level account assessment and recommendations. No installation required.
- **Checks** → examples: EBS/RDS public snapshots, root account usage, unrestricted security group rules.
- **Categories** → six groups: Cost optimization, Performance, Security, Fault tolerance, Service limits, Operational excellence.
- **Core vs Full checks** →
  - Core checks: available to all accounts (mainly security + service limits).
  - Full checks: require **Business** or **Enterprise Support Plan**.

## Hands-On Console Demo

- **Recommendations** → flagged two security issues:
  - Bucket allowing global access.
  - 29 security group rules with unrestricted ports.
- **Support plan upgrade** → needed to unlock cost optimization, performance, fault tolerance, and operational excellence checks.

- **Security checks** → available core checks include bucket permissions, security group ports, EBS/RDS public snapshots.
- **Service limits** → view limits for Auto Scaling Groups, CloudFormation stacks, DynamoDB capacity, etc.

### Amazon VPC Overview

- **Virtual Private Cloud** → private network to deploy AWS resources.
- **Subnets** →
  - Public subnets: accessible from the internet (e.g., websites).
  - Private subnets: isolated, no direct internet access (e.g., internal analytics).
- **High availability** → multiple subnets across multiple Availability Zones.

### Internet Access

- **Internet Gateway** → attaches to public subnets for inbound/outbound internet traffic.
- **NAT Gateway** → deployed in public subnet, allows private subnet instances outbound internet access (updates, downloads) without inbound exposure.

### Private Service Access

- **VPC Endpoints** → connect privately to AWS services without traversing the public internet.
- **AWS PrivateLink** → underlying technology enabling private connectivity to AWS services.
- **S3 Gateway Endpoint** → special endpoint for private access to Amazon S3.
- **Interface Endpoints** → alternative endpoint type for services like Bedrock, SageMaker, DynamoDB.

### IAM (Identity & Access Management)

- **IAM Users** → mapped to physical users, console login with password.
- **IAM Groups** → collections of users for easier permission management.
- **IAM Policies** → JSON documents defining permissions.
- **IAM Roles** → assigned to AWS services (e.g., EC2, Bedrock) for resource access.

### Compute Services

- **EC2** → virtual servers with OS/AMI, CPU, RAM, storage, security groups, and **User Data** for bootstrapping.
- **Lambda** → serverless functions, event-driven, auto-scaling, pay-per-use.

### Networking & Security

- **VPC Endpoints** → private access to AWS services via **PrivateLink**.
- **S3 Gateway Endpoint** → private access to S3 without internet exposure.

### Data Security & Compliance

- **Macie** → detects sensitive data (PII) in S3 buckets.
- **Config** → tracks resource configuration changes, compliance rules.
- **Inspector** → scans EC2, ECR images, and Lambda for vulnerabilities.
- **CloudTrail** → logs all API calls and user activity for audit.
- **Artifact** → provides compliance reports (ISO, PCI, SOC) and agreements (HIPAA, BAA).
- **Trusted Advisor** → account-level recommendations (cost, performance, security, limits).

### Bedrock-Specific Security Integrations

- **IAM with Bedrock** → identity verification, resource-level access control.
- **Guardrails** → restrict topics, filter harmful content, enforce safety policies.
- **CloudTrail with Bedrock** → logs API calls made to Bedrock.
- **Config with Bedrock** → tracks configuration changes (e.g., knowledge base updates).
- **PrivateLink with Bedrock** → keeps API calls private within VPC.

### Scenario 1: Bedrock accessing encrypted S3 data

- **Encryption** → S3 bucket encrypted with **SSE-KMS** using a KMS key.
- **IAM Role** → Bedrock customization/fine-tuning job must assume an IAM Role.
- **Permissions required:**
  - **S3 access** → read data from the bucket.
  - **KMS decrypt** → permission to decrypt objects using the KMS key.
- **Result** → Bedrock job can securely pull encrypted training data from S3.

### Scenario 2: SageMaker in VPC accessing S3 privately

- **Private subnets** → SageMaker notebooks/training jobs deployed in private subnets.
- **S3 VPC Endpoint** → enables private connectivity to S3 without internet exposure.
- **Security groups** → control inbound/outbound traffic.
- **Endpoint policy** → restricts which S3 buckets/resources can be accessed.
- **IAM Role** → attached to SageMaker resources, grants access to the endpoint.

### Scenario 3: Private access to Bedrock models

- **Bedrock VPC Endpoint** → created using **PrivateLink**.
- **Security group + endpoint policy** → regulate access.
- **Result** → application in private subnet can invoke Bedrock models without traversing the public internet.

### Scenario 4: Bedrock API calls tracked with CloudTrail

- **CloudTrail integration** → logs all API calls (successful or denied).
- **Authorized user** → CloudTrail shows successful invocation (e.g., ListCustomModels).
- **Unauthorized user** → CloudTrail records denied attempts, useful for auditing and detecting unauthorized access.